



FACULTAD DE MATEMÁTICAS

Trabajo fin de Grado

Grado en Matemáticas

Algoritmos de búsqueda en grafos: Generación de playlists de evolución musical

**Realizado por
Jacquelina Ruiz Dontsova**

**Dirigido por
Antonio Ramírez de Arellano Marrero y Andrés Nicolás Uranga Limón**

**Departamento
Ciencias de la Computación
e Inteligencia Artificial**

Sevilla, Mayo de 2026

Abstract

Graph theory makes it possible to model and solve a wide variety of problems. In this Bachelor's Thesis, graph theory and search algorithms are applied to a problem in the field of music: given two songs, generating a playlist that links them through musically coherent transitions. This problem is reformulated as the search for the shortest path in an undirected and weighted graph, whose vertices are songs and whose weights measure the difference between them.

The graph is built from a database of more than 30000 songs, after a preprocessing and cleaning phase. The edges of this graph are created using the k -Nearest Neighbors algorithm in an eight-feature space, with the Euclidean distance as metric and a genre compatibility matrix. On this graph, three algorithms are implemented and compared: Breadth-First Search (BFS), Dijkstra and A*. For A*, the heuristic used is proven to be admissible, which implies that the paths it returns are optimal. Moreover, the experimental comparison shows that it is the most efficient algorithm. The project concludes with an interactive application developed using the Streamlit library, which allows the user to generate playlists between any pair of songs in the graph and obtain the results of each algorithm.

Resumen

La teoría de grafos permite modelar y resolver una gran variedad de problemas. En este trabajo de fin de grado, se aplican la teoría de grafos y los algoritmos de búsqueda a un problema del ámbito musical: dadas dos canciones, generar una playlist que las una mediante transiciones musicalmente coherentes. Este problema se reformula como la búsqueda del camino más corto en un grafo no dirigido y ponderado, cuyos vértices son canciones y cuyos pesos miden la disimilitud entre ellas.

A partir de una base de datos de más de 30000 canciones se construye el grafo, tras una fase de preprocesamiento y limpieza. Las aristas de este grafo se crean mediante el algoritmo de los k vecinos más cercanos sobre un espacio de ocho características, empleando la distancia euclídea como métrica y una matriz de compatibilidad de géneros. Sobre este grafo se implementan y comparan tres algoritmos de búsqueda: Búsqueda por amplitud (BFS), Dijkstra y A*. Para A* se demuestra que la heurística empleada es admisible, por lo que las rutas que devuelve son óptimas. Además, la comparativa experimental muestra que es el algoritmo más eficiente. El proyecto finaliza con una aplicación interactiva creada con la librería Streamlit de Python, que permite generar playlists entre cualquier par de canciones del grafo y obtener los resultados de cada algoritmo.

Índice general

Índice general	V
Índice de figuras	VII
Índice de código	IX
1 Introducción	1
2 Conceptos teóricos	3
2.1 Fundamentos de la teoría de grafos	3
2.1.1 Primeras definiciones	3
2.1.2 Subgrafos	4
2.1.3 Caminos, circuitos y conectividad	5
2.1.4 Grafos ponderados	6
2.2 Análisis de complejidad algorítmica	7
2.2.1 Problemas y algoritmos	7
2.2.2 Concepto de eficiencia	7
2.2.3 Órdenes de complejidad	8
2.3 Estructuras de datos y representación computacional	8
2.3.1 Matriz de adyacencia	9
2.3.2 Listas de adyacencia	9
2.3.3 Criterio de selección	10
2.4 Construcción de grafos y espacios métricos	10
2.4.1 Distancias y espacios métricos	11
2.4.2 Algoritmo k -NN	12
2.5 Algoritmos de búsqueda no informada	13
2.5.1 Búsqueda por profundidad	13
2.5.2 Búsqueda por amplitud	14
2.5.3 Búsqueda de coste uniforme (Algoritmo de Dijkstra)	18
2.6 Algoritmos de búsqueda informada	24
2.6.1 Algoritmo A^*	24
3 Creación del grafo	31
3.1 Librerías y módulos utilizados	31
3.2 Descripción y preprocesamiento del dataset	32
3.3 Modelado del grafo	33
3.3.1 Elección de la métrica usada	34

3.3.2	Compatibilidad de géneros	35
3.3.3	Construcción de las aristas mediante k -NN	36
3.4	Representación computacional	37
3.5	Análisis del grafo final	37
4	Implementación de los algoritmos de búsqueda	39
4.1	Librerías y módulos utilizados	39
4.2	Breadth First Search (BFS)	40
4.2.1	Primera versión	40
4.2.2	Segunda versión	40
4.2.3	Tercera versión	41
4.3	Dijkstra	41
4.3.1	Primera versión	41
4.3.2	Segunda versión	42
4.4	A*	43
4.4.1	Justificación de la heurística elegida	43
5	Comparativas	45
5.1	Librerías y módulos utilizados	45
5.2	Criterios de comparación	46
5.3	Casos de estudio	46
5.4	BFS frente a Dijkstra	47
5.5	Dijkstra frente a A*	49
5.6	Comparativa global	51
5.7	Aplicación interactiva con Streamlit	52
6	Conclusiones	55
	Bibliografía	57

Índice de figuras

2.1	Grafo no dirigido (G) y grafo dirigido (G').	4
2.2	Grafo con lazo en el vértice a	4
2.3	Subgrafo H_1 de G obtenido al eliminar la arista (a, c) y subgrafo H_2 de G obtenido al eliminar el vértice c	5
2.4	Grafo ponderado.	6
2.5	Grafo G junto a su matriz de adyacencia A	9
2.6	Grafo G junto a su lista de adyacencia	9
2.7	Antes de k -NN [1]	12
2.8	Después de k -NN [1]	12
2.9	Grafo G	16
2.10	Grafo con pesos	21
2.11	Ruta más corta entre el vértice a y el vértice e	22
3.1	Gráfica grados	38
3.2	Gráfica pesos	38
5.1	Interfaz inicial	52
5.2	Playlist de BFS	53
5.3	Playlists de los tres algoritmos	54

Índice de código

4.1	Comprobación de vértices descubiertos sin el diccionario status	40
4.2	Detección anticipada del destino dentro del bucle for	41
4.3	Filtro para evitar procesar vértices repetidos	42
5.1	Cálculo del peso de la ruta obtenida con BFS	47

Introducción

La teoría de grafos es una de las áreas más versátiles de las matemáticas, con aplicaciones en ámbitos muy diversos. Entre las aplicaciones más relevantes se encuentra la optimización de rutas en redes de transporte, donde algoritmos como Dijkstra o A* calculan el camino más corto entre dos puntos del mapa. Sin embargo, su alcance va mucho más allá. En biología, los grafos permiten modelar las redes de interacción de proteínas, las relaciones evolutivas entre especies o la propagación de enfermedades infecciosas, entre otras muchas aplicaciones. En el ámbito social, los grafos describen las relaciones entre usuarios de una red, permitiendo detectar comunidades o comportamientos anómalos. En la inteligencia artificial, las redes neuronales tienen estructura de grafo, en el que los vértices son las neuronas artificiales y las aristas ponderadas son las conexiones entre ellas. Sobre este modelo se ha construido el aprendizaje profundo (deep learning). En definitiva, el modelado de problemas mediante la teoría de grafos puede aplicarse a problemas en los que la noción de distancia entre dos elementos no sea física o evidente, como, en este trabajo, la similitud entre canciones.

En este trabajo, se plantea una aplicación de la teoría de grafos en el ámbito musical. En concreto, se estudia el siguiente problema: dadas dos canciones, ¿es posible generar una secuencia de canciones musicalmente coherente entre ellas? Aplicado a teoría de grafos, este problema se puede reformular como la búsqueda del camino más corto entre dos canciones, cuya solución sería una playlist de evolución musical entre ambas.

A diferencia de otros problemas en los que el grafo viene dado de forma explícita, en este caso, no existía ninguna red que conectase las canciones entre sí. Para construir el grafo, se ha partido de una base de datos de más de 30000 canciones que proporciona varias características de cada canción (energía, bailabilidad, tempo...), además del género al que pertenece, para calcular la similitud entre canciones. Estos atributos y el sistema de compatibilidad de géneros que se explicará a lo largo del trabajo permiten la creación del grafo mediante el algoritmo de k -Nearest Neighbors (k vecinos más cercanos).

Sobre este grafo se han implementado y comparado tres algoritmos: Búsqueda por amplitud (BFS), Dijkstra y A*. La diferencia de objetivos y de rendimiento entre ellos permite analizar diferentes aspectos: la calidad de las transiciones, el número de canciones

que componen la playlist obtenida y la eficiencia de cada algoritmo.

Aunque el trabajo se basa principalmente en la teoría de grafos, se apoya también en herramientas de varias disciplinas. La construcción del grafo a partir de datos reales requiere una fase de ingeniería de datos. La noción de similitud entre canciones se formaliza utilizando la teoría de espacios métricos, mientras que la creación de las aristas se realiza mediante k -NN, que es un algoritmo de aprendizaje automático (machine learning). Para el estudio de los algoritmos de búsqueda se recurre al análisis de complejidad algorítmica. Por último, la aplicación interactiva aporta una componente de comunicación y visualización de los resultados.

Se comienza el trabajo explicando los conceptos teóricos que se necesitarán más adelante: fundamentos de teoría de grafos, análisis de complejidad algorítmica, estructuras de datos y representación computacional, noción de distancia y espacios métricos, el algoritmo de los k vecinos más cercanos y, por último, los algoritmos que se estudiarán en este trabajo.

A continuación, se describe el proceso de creación del grafo. Se explica primero el preprocesamiento de la base de datos, luego las decisiones clave para diseñar el modelo (elección de la métrica, compatibilidad de géneros, representación computacional) y, para acabar, se analiza el grafo obtenido para comprobar la calidad de su construcción.

Posteriormente, se detallan las implementaciones de los tres algoritmos, además de las sucesivas mejoras hasta llegar a la versión final. También se incluye la justificación de la heurística utilizada y una prueba de su admisibilidad.

El siguiente capítulo continúa con la comparativa entre los tres algoritmos sobre cuatro pares de canciones del grafo construido, bajo diferentes métricas (tiempo, vértices explorados, coste de la ruta y número de saltos). El capítulo concluye con la implementación de una aplicación interactiva desarrollada con la librería **Streamlit**, que permite generar playlists de transición para cualquier par de canciones del grafo.

Para finalizar, se presentan las conclusiones del trabajo y se proponen posibles mejoras.

Conceptos teóricos

En este capítulo se presentan los conceptos teóricos necesarios para el trabajo. Se comienza introduciendo los fundamentos de teoría de grafos y el análisis de complejidad algorítmica. Más tarde se describen las estructuras de datos usadas para representar grafos. A continuación, se definen las métricas empleadas para medir la similitud entre elementos de un espacio métrico y se explica el algoritmo de k -Nearest Neighbors, usado en este trabajo para la creación de aristas de un grafo. Finalmente, se presentan los algoritmos de búsqueda (Depth First Search, Breadth First Search, Dijkstra y A*).

2.1– Fundamentos de la teoría de grafos

En esta sección, se establecen los conceptos base de grafos siguiendo [7].

2.1.1. Primeras definiciones

Definición 2.1 (Grafo no dirigido). Un grafo es un par $G = (V, E)$, donde:

- V es un conjunto finito no vacío cuyos elementos son llamados **vértices** o **nodos**.
- $E \subseteq \{(a, b) : a, b \in V\}$ es un conjunto formado por pares no ordenados de vértices cuyos elementos son llamados **aristas**. Como el par es no ordenado se tiene que $(a, b) = (b, a)$, $\forall a, b \in V$.

A este tipo de grafo también se le denomina **grafo no dirigido**. Si consideramos al conjunto de aristas E como un conjunto de pares ordenados de vértices, diremos que G es un **grafo dirigido**.

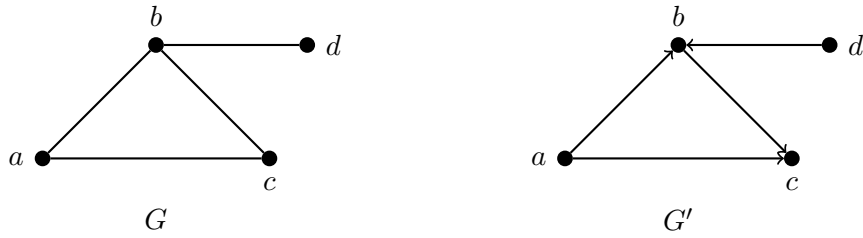


Figura 2.1: Grafo no dirigido (G) y grafo dirigido (G').

A efectos de este trabajo, consideraremos que el grafo G es no dirigido. Esta simplificación facilita el análisis de los algoritmos y es coherente con la simetría de las métricas de distancia que utilizaremos.

Definición 2.2 (Lazos y grafos simples). Las aristas de la forma (v, v) son llamadas **lazos**. Un grafo no dirigido libre de lazos es llamado **grafo simple**.

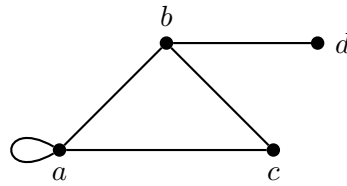


Figura 2.2: Grafo con lazo en el vértice a.

Definición 2.3 (Vértices adyacentes y aristas incidentes). Decimos que dos vértices u, v de un grafo G son **adyacentes** si existe una arista $e = (u, v)$ que los una. En este caso, diremos que u y v son extremos de e y que la arista e es **incidente** a u y v .

Definición 2.4 (Grado de un vértice). El **grado** de un vértice v en un grafo G no dirigido es igual al número de aristas en G que son incidentes a v . Lo denotaremos $\deg(v)$.

Definición 2.5 (Vértice aislado y grafo trivial). Un vértice de grado cero es llamado **vértice aislado**. Un grafo formado por un único vértice, y por tanto sin aristas, es llamado **grafo trivial**.

2.1.2. Subgrafos

Definición 2.6 (Subgrafo). Sea G un grafo. Un grafo H es llamado un **subgrafo** de G si los vértices y aristas de H están contenidas en los conjuntos de vértices y aristas de G .

Se pueden obtener subgrafos de un grafo eliminando vértices o aristas. Si e es una arista del grafo G , denotamos $G \setminus e$ al grafo obtenido al eliminar e , aunque sus extremos se mantienen.

Del mismo modo, si v es un vértice de G , denotamos $G \setminus v$ al grafo obtenido al eliminar v junto a las aristas incidentes a v .

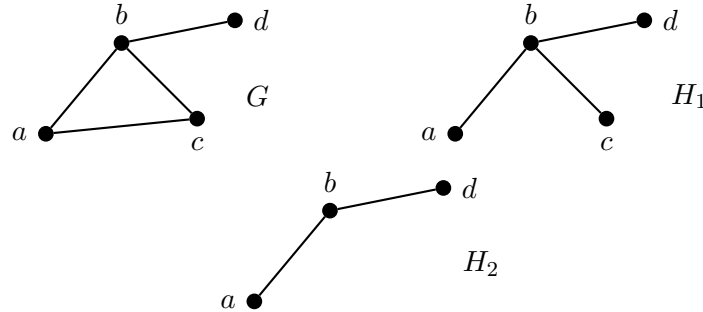
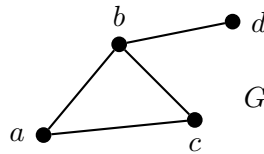


Figura 2.3: Subgrafo H_1 de G obtenido al eliminar la arista (a, c) y subgrafo H_2 de G obtenido al eliminar el vértice c .

2.1.3. Caminos, circuitos y conectividad

Consideremos de nuevo el grafo G :



Definición 2.7 (Camino). Un **camino** en un grafo G es una sucesión finita de vértices $(v_0, v_1, \dots, v_{n-1}, v_n)$, donde $(v_i, v_{i+1}) \in E$. Llamamos **longitud del camino** al número de aristas que lo forman.

Por ejemplo, (a, b, d) es un camino en G 2.1.3 de longitud 2.

Definición 2.8 (Camino cerrado). Se dice que un camino es **cerrado** si $v_0 = v_n$.

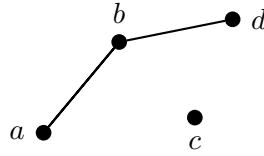
Definición 2.9 (Recorrido). Un camino en el que todas las aristas son distintas es llamado **recorrido**.

Definición 2.10 (Camino simple). Un **camino simple** es aquel en el que todos sus vértices son distintos.

Definición 2.11 (Ciclo). Si el camino es simple y además cerrado decimos que es un **ciclo**.

Definición 2.12 (Grafo conexo). Un grafo G es **conexo** si existe un camino entre cada par de vértices. En caso contrario, diremos que G es **disconexo**.

El grafo G 2.1.3 es conexo. Mientras que el siguiente grafo no lo es.



Definición 2.13 (Componente conexa). Se denomina **componente conexa** de un grafo G a todo subgrafo conexo maximal de G , es decir, un subgrafo conexo que no está contenido en ningún otro subgrafo conexo más grande de G .

Definición 2.14 (Punto de corte). Un vértice v es llamado un **punto de corte** si $G \setminus v$ es un grafo desconexo, es decir, si al eliminarlo aumenta el número de componentes conexas de G .

Definición 2.15 (Puente). Una arista e es un **punto de corte** si $G \setminus e$ es un grafo desconexo, es decir, al eliminarla aumenta el número de componentes conexas de G .

En el grafo G 2.1.3, se tiene que el vértice b es un punto de corte y la arista (b, d) es un puente.

Aunque el vértice b es extremo de un puente y a la vez un punto de corte, en general, los extremos de un puente no son necesariamente puntos de corte. Por ejemplo, en G , tenemos que el vértice d es extremo del puente (b, d) pero no es un punto de corte, ya que al eliminarlo el grafo sigue siendo conexo.

2.1.4. Grafos ponderados

Definición 2.16 (Grafo ponderado). Un grafo G es llamado **grafo ponderado** si a cada arista e de G le es asignado un número no negativo $w(e)$ llamado **peso** de e .

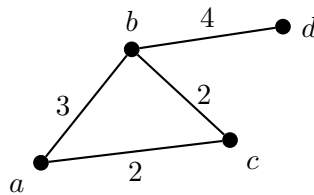


Figura 2.4: Grafo ponderado.

Definición 2.17 (Peso de un camino). El **peso de un camino** $P = (v_0, v_1, \dots, v_{n-1}, v_n)$ en un grafo ponderado G es definido como la suma de los pesos de las aristas que lo constituyen:

$$w(P) = \sum_{i=1}^n w(v_{i-1}, v_i)$$

Por ejemplo, el peso del camino $P = (a, c, b, d)$ en el grafo G' 2.4 es de 8.

2.2– Análisis de complejidad algorítmica

En esta sección se presentan los conceptos fundamentales para el análisis de algoritmos incluyendo la definición de problema y algoritmo, el concepto de eficiencia y órdenes de complejidad, siguiendo a [2], [10] y [11].

2.2.1. Problemas y algoritmos

Definición 2.18 (Problema computacional). Un **problema computacional** es una relación entre un conjunto de valores de entrada y un conjunto de valores de salida. La formulación del problema especifica la relación deseada entre la entrada y la salida, es decir, las condiciones que debe cumplir la entrada y las propiedades que debe satisfacer la salida en función de dicha entrada.

Definición 2.19 (Algoritmo). Un **algoritmo** es un procedimiento computacional bien definido que toma cierto valor o conjunto de valores como entrada y produce algún valor o conjunto de valores como salida. Por tanto, un algoritmo es una secuencia de pasos computacionales que transforma valores de entrada en valores de salida.

Un algoritmo se dice **correcto** si para toda entrada que satisface las condiciones del problema, el algoritmo se detiene y produce el resultado correcto. Decimos que un algoritmo correcto resuelve el problema computacional dado.

2.2.2. Concepto de eficiencia

Una vez definido el concepto de algoritmo, surge la pregunta de cómo de eficiente es un algoritmo para resolver un problema dado. En capítulos siguientes se verá que dos algoritmos correctos que resuelvan el mismo problema pueden diferir enormemente en eficiencia.

El análisis de eficiencia de un algoritmo consiste en medir la cantidad de recursos necesarios para su ejecución. Para ello, cuantificamos la cantidad de operaciones elementales realizadas en función del tamaño de la entrada.

En este contexto, una **operación elemental** es una operación cuyo coste se considera constante, es decir, que no depende del tamaño del problema. Ejemplos de operaciones elementales son las asignaciones de variables, las comparaciones entre valores y las operaciones aritméticas básicas.

En el caso de la teoría de grafos, el tamaño de la entrada se define con dos parámetros: número de vértices ($|V|$ o n) y número de aristas ($|E|$ o m). El objetivo es obtener una función $T(n, m)$ que represente el número de operaciones elementales en el peor de los casos, garantizando así un límite superior para cualquier instancia del problema.

2.2.3. Órdenes de complejidad

En los estudios de complejidad de algoritmos, se asumirá que las funciones $T(n, m)$ son no negativas y crecientes, y solo será interesante estudiar su comportamiento para grandes valores de n y m . Por lo que la clasificación de estas funciones se hará según su crecimiento asintótico, agrupándolas en órdenes de complejidad.

El **orden de complejidad** de una función se denota como $T(n, m) = \mathcal{O}(g(n, m))$.

Definición 2.20 (Big-O). Sean $T, g : \mathbb{N}^2 \rightarrow \mathbb{R}_{\geq 0}$. Decimos que $T(n, m) = \mathcal{O}(g(n, m))$ si existen constantes $c > 0$ y $n_0, m_0 \in \mathbb{N}$ tales que:

$$T(n, m) \leq c \cdot g(n, m), \quad \forall n \geq n_0, \forall m \geq m_0.$$

Intuitivamente, esto significa que el orden de crecimiento de $T(n, m)$ no es mayor que el orden de crecimiento de $g(n, m)$.

Se define también el **orden de complejidad exacto**, denotado como $T(n, m) = \Theta(g(n, m))$.

Definición 2.21 (Big-theta). Sean $T, g : \mathbb{N}^2 \rightarrow \mathbb{R}_{\geq 0}$. Decimos que $T(n, m) = \Theta(g(n, m))$ si existen constantes positivas C_1 y C_2 y $n_0, m_0 \in \mathbb{N}$ tales que

$$C_1 g(n, m) \leq T(n, m) \leq C_2 g(n, m), \quad \forall n \geq n_0, \forall m \geq m_0.$$

Intuitivamente, esto significa que $T(n, m)$ crece asintóticamente al mismo ritmo que $g(n, m)$.

Ahora, se definen dos tipos de complejidad de un algoritmo.

Definición 2.22. Sea A un algoritmo. Se define:

- La **complejidad temporal** de A es el número de operaciones elementales que realiza a partir de los datos de entrada.
- La **complejidad espacial** de A es la cantidad de memoria requerida (suma total del espacio que ocupan las variables del algoritmo) antes, durante y después de su ejecución.

2.3— Estructuras de datos y representación computacional

Para poder aplicar un algoritmo a un grafo $G = (V, E)$, necesitamos representar este grafo mediante una estructura de datos que nos permita realizar consultas y modificaciones de forma eficiente. En la práctica, las dos estructuras de datos más utilizadas son matrices de adyacencia y listas de adyacencia. Seguiremos en esta sección a [3].

2.3.1. Matriz de adyacencia

Definición 2.23. La **matriz de adyacencia** de un grafo G no dirigido es una matriz booleana A de dimensión $|V| \times |V|$, en la que cada entrada indica si una arista concreta está en G . En particular, para todos los vértices u y v :

$$A[u, v] := 1 \text{ si y solo si } (u, v) \in E$$

Para grafos no dirigidos, la matriz A siempre es simétrica ($A[u, v] = A[v, u]$) y las entradas de la diagonal principal ($A[u, u]$) son todas cero, ya que no se consideran lazos.

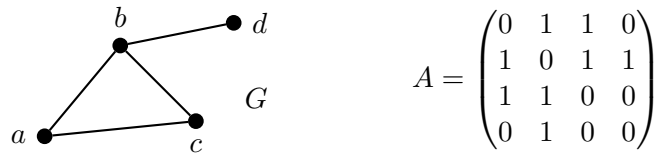


Figura 2.5: Grafo G junto a su matriz de adyacencia A

Para grafos ponderados, la entrada $A[u, v]$ almacena el peso de la arista en lugar de un valor booleano.

Las matrices de adyacencia requieren un espacio $\mathcal{O}(|V|^2)$, sin importar cuántas aristas tenga el grafo, así que solo son eficientes para grafos densos.

Dada una matriz de adyacencia, se puede decidir en tiempo $\mathcal{O}(1)$ si dos vértices están conectados mirando la entrada apropiada en la matriz. Mientras que enumerar todos los vecinos de un vértice requiere tiempo $\mathcal{O}(|V|)$ escaneando la fila (o columna) correspondiente.

2.3.2. Listas de adyacencia

Definición 2.24. Una **lista de adyacencia** es un vector de listas (array of lists), cada una conteniendo los vecinos de uno de los vértices. En el caso de un grafo no dirigido, cada arista (u, v) es almacenada dos veces (una en la lista de vecinos de u y otra en la lista de vecinos de v).

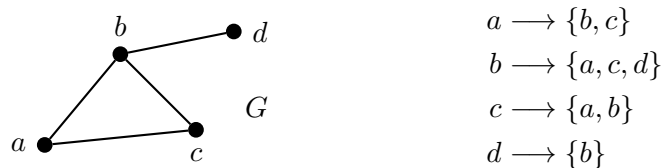


Figura 2.6: Grafo G junto a su lista de adyacencia

Para representar los pesos en un grafo ponderado, cada elemento de la lista almacena un par (vértice, peso).

Sin importar el tipo de grafo, el espacio requerido para una lista de adyacencia es $\mathcal{O}(|V| + |E|)$.

Dada una lista de adyacencia, se pueden enumerar los vecinos de un vértice v en un tiempo $\mathcal{O}(1 + \deg(v))$, solo escaneando la lista de los vecinos de v . De la misma forma, podemos determinar si (u, v) es una arista en tiempo $\mathcal{O}(1 + \min\{\deg(u), \deg(v)\})$, escaneando simultáneamente las listas de los vecinos de u y v , y parando cuando encontremos la arista o cuando llegamos al final de una lista.

2.3.3. Criterio de selección

La siguiente tabla resume el rendimiento de ambas estructuras de datos basándose en el número de vértices (V) y de aristas (E).

Operación	Matriz de adyacencia	Lista de adyacencia
Espacio de memoria	$\mathcal{O}(V ^2)$	$\mathcal{O}(V + E)$
Test de adyacencia $(u, v) \in E$	$\mathcal{O}(1)$	$\mathcal{O}(1 + \min\{\deg(u), \deg(v)\})$
Listar vecinos de un vértice u	$\mathcal{O}(V)$	$\mathcal{O}(1 + \deg(u))$
Listar todas las aristas	$\mathcal{O}(V ^2)$	$\mathcal{O}(V + E)$

Tabla 2.1: Comparativa de operaciones básicas según la representación

Como se puede deducir a partir de la tabla 2.1, no existe una estructura superior en todos los ámbitos, sino que se usará la más conveniente dependiendo de la densidad del grafo:

- **Grafos densos:** Si el número de aristas $|E|$ es cercano a $|V|^2$, es preferible la matriz de adyacencia, ya que son más simples y más eficientes.
- **Grafos dispersos:** En la mayoría de casos prácticos, el número de aristas es mucho menor a $|V|^2$, por lo que las listas de adyacencia son significativamente más eficientes, tanto en espacio como en el tiempo necesario para explorar los vecinos de un vértice.

2.4— Construcción de grafos y espacios métricos

En diversas aplicaciones, el grafo no viene dado explícitamente, sino que debe construirse a partir de un conjunto de objetos situados en un espacio métrico. Para ello, se representará cada objeto como un vector en $\mathbb{R}^n$, en el que cada componente corresponde a una característica medible del objeto.

2.4.1. Distancias y espacios métricos

Definición 2.25. Una **distancia métrica** sobre un conjunto X es una aplicación $d : X \times X \rightarrow [0, +\infty)$ que cumple las siguientes propiedades:

- $d(x, y) = 0$ si y solo si $x = y$.
- $d(x, y) = d(y, x)$, para cualesquiera $x, y \in X$ (propiedad simétrica).
- $d(x, y) \leq d(x, z) + d(z, y)$, para cualesquiera $x, y, z \in X$ (desigualdad triangular).

El par (X, d) se denomina **espacio métrico**.

Dependiendo de la naturaleza de los datos, se elegirá una métrica u otra para determinar la distancia entre dos vectores. Sea $X = \mathbb{R}^n$ y sean $x, y \in \mathbb{R}^n$, tenemos, por ejemplo:

1. La aplicación

$$d_e((x_1, \dots, x_n), (y_1, \dots, y_n)) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

es la llamada **distancia euclídea** sobre $\mathbb{R}^n$.

2. La aplicación

$$d_t(x, y) = \sum_{i=1}^n |x_i - y_i|$$

es la llamada **taxi-distancia**.

3. La aplicación

$$d_M(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

con Σ la matriz de covarianza, es la llamada **distancia de Mahalanobis**.

Fuera de las métricas puras, existe la **similitud del coseno**. A diferencia de las anteriores, esta no mide la magnitud de los vectores, sino el coseno del ángulo entre ellos. Aunque es muy utilizada en la práctica computacional por su invarianza ante la escala, es una medida de disimilitud, ya que no siempre satisface la desigualdad triangular:

$$d_{cos}(x, y) = 1 - \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$

2.4.2. Algoritmo k -NN

El algoritmo de los k vecinos más cercanos (k-nearest neighbors o k -NN) es un método de aprendizaje automático supervisado, utilizado fundamentalmente para resolver problemas de clasificación y de regresión. La idea detrás de k -NN es que se asume que las observaciones con características similares tienden a estar cerca unas de otras. Dado un entero positivo k y una observación de prueba X_0 , el algoritmo identifica los k puntos en el conjunto de datos de entrenamiento que están más cerca de X_0 . En problemas de clasificación, k -NN estima la probabilidad de cada categoría y le asigna a X_0 la clase con mayor probabilidad. Para problemas de regresión, la predicción se realiza calculando la media de los valores de los k vecinos más cercanos [5].

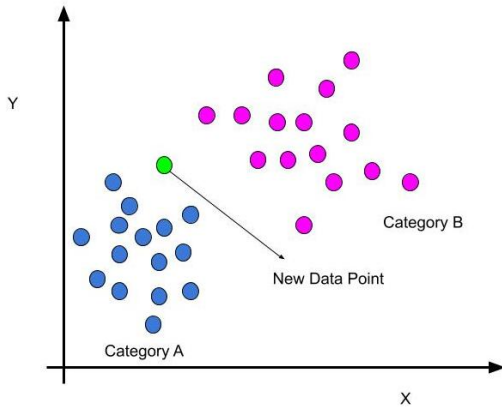


Figura 2.7: Antes de k -NN [1]

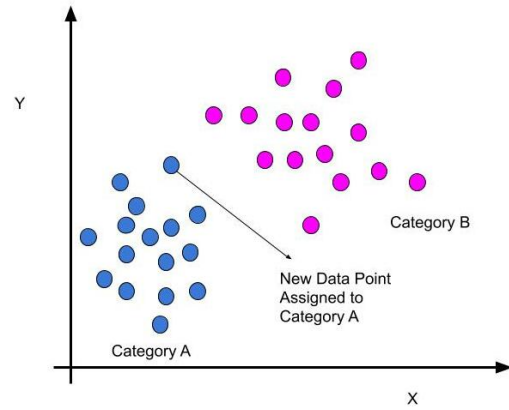


Figura 2.8: Después de k -NN [1]

En este trabajo, k -NN se emplea de forma no supervisada: no se utilizará para predecir etiquetas, sino que se aprovechará la lógica de proximidad espacial para construir un grafo a partir de un conjunto de objetos situados en un espacio métrico (X, d) .

Dado un espacio métrico (X, d) , un conjunto de vértices $V \subset X$ y un entero $k \geq 1$, se sigue el siguiente procedimiento para cada vértice $v \in V$:

1. Se calcula la distancia $d(v, u)$ respecto a todos los demás vértices $u \in V \setminus \{v\}$.
2. Se ordenan estas distancias de menor a mayor.
3. Se define su conjunto de vecinos $\mathcal{N}_k(v)$ como los k vértices asociados a las menores distancias.
4. Se establece una relación (arista dirigida) desde v hacia cada uno de los vértices en $\mathcal{N}_k(v)$.

Nótese que la relación de “ser vecino” no es necesariamente simétrica, es decir, $u \in \mathcal{N}_k(v)$ no implica que $v \in \mathcal{N}_k(u)$. Así que, para obtener un grafo no dirigido y que los algoritmos funcionen correctamente, se aplicará la siguiente estrategia de simetrización: Si $u \in \mathcal{N}_k(v)$ o $v \in \mathcal{N}_k(u)$, se crea la arista no dirigida (u, v) .

2.5– Algoritmos de búsqueda no informada

Definición 2.26. Un **algoritmo de búsqueda** es un conjunto de instrucciones diseñadas para localizar un cierto elemento en una estructura de datos, en nuestro caso, en un grafo.

A continuación, se presentarán diferentes algoritmos de búsqueda que se consideran importantes para este trabajo. En esta sección sobre algoritmos de búsqueda no informada, se seguirá [10] para la descripción de los algoritmos y [2] para la complejidad y las pruebas de corrección.

Los algoritmos de búsqueda no informada son estrategias de búsqueda en los que no se dispone de información adicional sobre qué camino podría ser mejor.

2.5.1. Búsqueda por profundidad

Aunque se presenta el algoritmo de Búsqueda por profundidad (DFS) por su importancia teórica y a modo de contraste con la Búsqueda por amplitud, no se profundizará en él ni se usará en la parte experimental. El motivo de esta decisión es que DFS no garantiza encontrar el camino más corto, ni en coste ni en número de saltos, por lo que no resulta adecuado para el objetivo de este trabajo (generar la transición musical más coherente entre dos canciones).

El algoritmo de **búsqueda por profundidad** examina los vértices y aristas de un grafo G de manera sistemática, priorizando la exploración de rutas completas antes de retroceder. La estrategia consiste en procesar un vértice inicial A y continuar explorando un vecino de A , luego un vecino de este último y así sucesivamente, alejándose del origen tanto como sea posible hasta alcanzar un “punto muerto”, es decir, un vértice sin vecinos no procesados.

Durante la ejecución del algoritmo, cada vértice X estará en uno de estos tres estados:

- $STATUS(X) = 1$: El vértice X **no** ha sido **procesado**.
- $STATUS(X) = 2$: El vértice X ha sido **descubierto** y está en lista de espera.
- $STATUS(X) = 3$: El vértice X ha sido **procesado**.

La lista de espera usada en este algoritmo será una pila (stack), que es una estructura de datos que sigue el principio LIFO (Last-In, First-Out). El procedimiento se describe de la siguiente manera:

1. Se inicializa una pila vacía y se marcan todos los vértices como no procesados.
2. Se toma el vértice inicial A , se marca como descubierto y se introduce en la pila.
3. Mientras la pila no esté vacía:

- Se extrae el último vértice X que fue añadido a la pila.
- Se marca el vértice X como procesado y se calculan todos sus vecinos.
- Para cada vecino de X que no haya sido procesado, se actualiza su estado a descubierto y se inserta en la pila.

El algoritmo concluye cuando la pila queda vacía, lo que significa que se han explorado todos aquellos vértices conectados al vértice inicial A .

2.5.2. Búsqueda por amplitud

La **Búsqueda por Amplitud (BFS)** propone una estrategia de exploración por capas. A diferencia del enfoque en profundidad, este algoritmo procesa primero todos los vecinos directos del vértice inicial antes de proceder con los vecinos de estos. La gestión de los estados $STATUS(X)$ se mantiene idéntica a la descrita anteriormente. Sin embargo, la diferencia estructural radica en el uso de una cola (queue) para la lista de espera, que sigue el principio FIFO (First-In, First-Out). El procedimiento se describe de la siguiente manera:

1. Se inicializa una cola vacía Q y se marcan todos los vértices como no procesados.
2. Se toma el vértice inicial A , se actualiza su estado a descubierto y se añade a la cola.
3. Mientras la cola no esté vacía:
 - Se desencola el primer vértice X que fue añadido a la cola.
 - Se marca el vértice X como procesado y se obtienen sus vecinos.
 - Para cada vecino de X que se encuentre en estado no procesado, se actualiza su estado a descubierto y se encola.

Complejidad

La complejidad temporal del algoritmo BFS es $\mathcal{O}(|V| + |E|)$. Después de la inicialización, cada vértice se encola y desencola como máximo una vez. Como las operaciones de encolar y desencolar toman un tiempo $\mathcal{O}(1)$, el total de tiempo invertido en las colas es $\mathcal{O}(|V|)$. Además, como la lista de adyacencia de cada vértice se explora únicamente cuando este es desencolado, cada lista de adyacencia se escanea como mucho una vez. Como la suma de las longitudes de todas las listas de adyacencia es $\Theta(|E|)$, el tiempo total invertido en escanear listas de adyacencia es $\mathcal{O}(|E|)$. Así que sumando ambos costes, se tiene que el tiempo de ejecución total es $\mathcal{O}(|V| + |E|)$.

La complejidad espacial es $\mathcal{O}(|V|)$. El consumo de memoria está dominado por la estructura de datos (la cola) utilizada para almacenar los vértices descubiertos que están a la espera de ser procesados. En el peor de los casos, la cola almacenará a todos los vértices del grafo simultáneamente (por ejemplo, si todos los nodos están conectados directamente al nodo de origen), por lo que el espacio máximo requerido está acotado por el número de vértices.

Pseudocódigo

En el pseudocódigo $d[v]$ representa la longitud del camino (número de aristas) desde el vértice de origen A hasta v , y $parent[v]$ almacena el predecesor inmediato de v en dicho camino.

A continuación se muestra un pseudocódigo del algoritmo BFS:

Algoritmo 1: BFS

```
BreadthFirstSearch(graph  $G$ , vertex  $A$ )
1:  $Q = \emptyset$ 
2: for  $v \in V(G) \setminus \{A\}$  do
3:    $status[v] = 1$ 
4:    $parent[v] = \text{None}$ 
5:    $d[v] = \infty$ 
6: end-of-for
7:  $status[A] = 2$ 
8:  $parent[A] = \text{None}$ 
9:  $\text{Enqueue}(Q, A)$ 
10:  $d[A] = 0$ 
11: while  $Q \neq \emptyset$  do
12:    $u = \text{Dequeue}(Q)$ 
13:    $status[u] = 3$ 
14:    $N(u) = \text{getNeighborhood}(u)$ 
15:   for each  $v \in N(u)$  do
16:     if  $status[v] == 1$  then
17:        $status[v] = 2$ 
18:        $parent[v] = u$ 
19:        $d[v] = d[u] + 1$ 
20:        $\text{Enqueue}(Q, v)$ 
21:     end-of-if
22:   end-of-for
23: end-of-while
```

Ejemplo de BFS

Considérese el siguiente grafo, al que a continuación se le aplicará el algoritmo de Búsqueda en Amplitud iniciando por el vértice a :

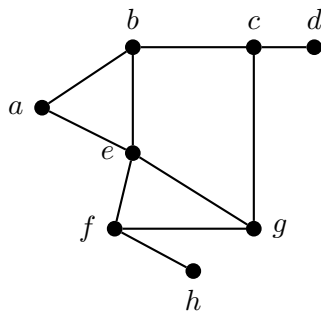


Figura 2.9: Grafo G

En la siguiente tabla se muestra el estado de cada vértice en cada iteración t del algoritmo y en la tabla 2.3 se muestra cómo evoluciona el estado de la lista de espera.

Vértice	$t = 0$	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$	$t = 6$	$t = 7$	$t = 8$
a	2	3	3	3	3	3	3	3	3
b	1	2	3	3	3	3	3	3	3
c	1	1	2	2	3	3	3	3	3
d	1	1	1	1	2	2	2	3	3
e	1	2	2	3	3	3	3	3	3
f	1	1	1	2	2	3	3	3	3
g	1	1	1	2	2	2	3	3	3
h	1	1	1	1	1	2	2	2	3

Tabla 2.2: Procesamiento de vértices del grafo 2.9 en cada iteración de BFS

t	Vértice procesado	Cola
0		a
1	a	b, e
2	b	e, c
3	e	c, f, g
4	c	f, g, d
5	f	g, d, h
6	g	d, h
7	d	h
8	h	

Tabla 2.3: Estado de la cola en cada iteración de BFS

Corrección

Antes de proceder con la demostración de la prueba de corrección del algoritmo BFS, es necesario definir la distancia $\delta(u, v)$ entre dos vértices u y v .

Definición 2.27. Definimos la distancia del camino más corto $\delta(u, v)$ desde un vértice u a otro vértice v como el número mínimo de aristas de cualquier camino que conecte a u y a v . Si no existe ningún camino que los conecte, $\delta(u, v) = \infty$.

Los siguientes lemas previos serán necesarios para la demostración del Teorema de Corrección 2.3

Lema 2.1. Sea $G = (V, E)$ un grafo dirigido o no dirigido, y supongamos que BFS es ejecutado en G desde un vértice origen $A \in V$ dado. Al finalizar, para cada vértice $v \in V$, el valor $d[v]$ calculado por BFS satisface $d[v] \geq \delta(A, v)$.

Corolario 2.2. Supongamos que los vértices v_i y v_j son encolados durante la ejecución de BFS, y que v_i es encolado antes de v_j . Entonces, $d[v_i] \leq d[v_j]$ en el momento en el que v_j es encolado.

Se puede demostrar ahora la corrección del algoritmo BFS. Para esta demostración, se usará el pseudocódigo descrito anteriormente.

Teorema 2.3. Sea $G = (V, E)$ un grafo dirigido o no dirigido, y supongamos que BFS es ejecutado en G desde un vértice origen $A \in V$ dado. Entonces, durante su ejecución, BFS descubre cada vértice $v \in V$ que es alcanzable desde el origen A , y al finalizar, $d[v] = \delta(A, v)$ para todo $v \in V$. Además, para cada vértice $v \neq A$ que es alcanzable desde A , uno de los caminos más cortos desde A a v es un camino más corto desde A a $\text{parent}[v]$ seguido de la arista $(\text{parent}[v], v)$.

Demostración. Asumamos, por reducción al absurdo, que algún vértice recibe un valor d diferente a la distancia del camino más corto. Sea v el vértice con el mínimo $\delta(A, v)$ que recibe ese valor incorrecto d . Por el lema 2.1, $d[v] \geq \delta(A, v)$, y por tanto tenemos $d[v] > \delta(A, v)$. El vértice v debe ser alcanzable desde A , porque si no es así, entonces $d[v] = \infty \geq \delta(A, v)$. Sea u el vértice inmediatamente anterior a v en un camino más corto de A a v , de modo que $\delta(A, v) = \delta(A, u) + 1$. Porque $\delta(A, u) < \delta(A, v)$, y por cómo hemos elegido v , tenemos $d[u] = \delta(A, u)$. Uniendo estas propiedades, tenemos:

$$d[v] > \delta(A, v) = \delta(A, u) + 1 = d[u] + 1 \quad (2.1)$$

Ahora consideremos el momento en el que BFS elige desencolar el vértice u de Q en la línea 11 de 2.5.2. En este momento, el vértice v está en estado $STATUS[v] = 1, 2$ ó 3 . En cada uno de los tres casos, llegamos a una contradicción con la desigualdad (2.1).

- Si $STATUS[v] = 1$: Por la línea 19, $d[v] = d[u] + 1$, contradiciendo la desigualdad (2.1).

- Si $STATUS[v] = 2$: Se puso $STATUS[v] = 2$ al desencolar algún vértice w , que fue eliminado de Q antes que u y por el cual $d[v] = d[w] + 1$. Por el corolario 2.2, sin embargo, $d[w] \leq d[u]$, por lo que tenemos $d[v] = d[w] + 1 \leq d[u] + 1$, contradiciendo de nuevo a la desigualdad (2.1).
- Si $STATUS[v] = 3$: Entonces, v ya había sido desencolado y, por el corolario 2.2, tenemos $d[v] \leq d[u]$, de nuevo contradiciendo la desigualdad (2.1).

Por lo tanto, se concluye que $d[v] = \delta(A, v)$ para todo $v \in V$. Todos los vértices v alcanzables desde A deben ser descubiertos, de lo contrario tendrían $\infty = d[v] > \delta(A, v)$. Para concluir la demostración, obsérvese que si $parent[v] = u$, entonces $d[v] = d[u] + 1$. Por tanto, se puede obtener un camino más corto desde A hasta v tomando un camino más corto desde A hasta $parent[v]$ y luego atravesando la arista $(parent[v], v)$. $\square$

2.5.3. Búsqueda de coste uniforme (Algoritmo de Dijkstra)

A diferencia de las búsquedas por amplitud y por profundidad, que asumen que los costes de cada paso son idénticos, la búsqueda de coste uniforme está diseñada para trabajar con grafos ponderados.

La búsqueda de coste uniforme no se preocupa por el número de pasos que tiene un camino, pero sí por su coste total. Este algoritmo se asegura de que el próximo vértice procesado sea aquel con el menor coste acumulado desde el origen.

Sea $G = (V, E)$ un grafo ponderado con pesos no negativos, nuestro objetivo es encontrar el peso de la ruta más corta entre dos vértices dados u y v , que en este contexto, redefinimos como sigue.

Definición 2.28. Definimos el peso de la ruta más corta en grafos ponderados como:

$$\delta(u, v) = \begin{cases} \min\{w(P) : P = (u, \dots, v)\} & \text{Si existe una ruta de } u \text{ a } v. \\ \infty & \text{Si no existe una ruta de } u \text{ a } v. \end{cases}$$

El algoritmo de Dijkstra asigna una distancia inicial infinita a todos los vértices del grafo, menos al vértice de inicio A , al que se le asigna el valor 0. Posteriormente, recorre los caminos que parten de A y mejora las distancias de sus vértices, mediante la técnica de **relajación**, eligiendo en cada iteración el vértice no procesado con menor distancia tentativa.

El procedimiento se describe de la siguiente manera:

1. A cada vértice v se le asigna un valor $d[v]$, llamado estimado de la ruta más corta, que es una cota superior del peso de la ruta más corta desde el vértice inicial A a v . Este valor también se denomina distancia tentativa. Inicialmente se asigna $d[A] = 0$ para el vértice de origen y $d[v] = \infty$ para el resto de vértices. Además, se inicializa un registro de predecesores $parent$, donde $parent[v] = None$ para todo vértice v .

2. Se inicializa el conjunto de vértices T con todos los vértices del grafo. Este conjunto contendrá en cada momento los vértices que aún no hayan sido procesados.
3. Se inicializa un conjunto $procesados = \emptyset$ para controlar los vértices cuyo camino mínimo ya ha sido garantizado y, por tanto, no deben volver a ser evaluados.
4. Mientras $T \neq \emptyset$:
 - Se extrae de T el vértice u que posea la menor distancia tentativa acumulada en ese instante y se añade a $procesados$.
 - Si el vértice u extraído coincide con el vértice destino, se detiene la exploración.
 - Se recorren todos los vértices adyacentes a u que todavía no pertenezcan al conjunto $procesados$. Para cada uno de estos vecinos v , se evalúa si la ruta conocida hacia v puede ser mejorada pasando a través de u . Para ello, comprobamos la siguiente condición de relajación:
 - * Si $d[v] > d[u] + w(u, v)$, se actualiza la distancia mínima $d[v] = d[u] + w(u, v)$ y se registra a u como su predecesor ($parent[v] = u$).
 - * En caso contrario, se mantiene el valor actual de $d[v]$ y de $parent[v]$.

Al finalizar, si el vértice destino tiene una distancia $d[\text{destino}] \neq \infty$, se utiliza el registro de predecesores $parent$ para trazar la ruta desde el destino hacia el origen. Si la distancia es ∞ , concluimos que no existe un camino viable entre ambos vértices.

Complejidad

La complejidad temporal del algoritmo de Dijkstra depende fundamentalmente de la estructura de datos utilizada para implementar la cola de prioridad. En la cola de prioridad, el algoritmo realiza tres tipos de operaciones: inserción, extracción del mínimo y relajación. Para cada vértice, el algoritmo realiza una operación de inserción y extracción del mínimo. Por lo que se realizan $|V|$ operaciones de inserción y de extracción. Además, como cada arista se examina una vez, la operación de relajación se ejecuta, en el peor de los casos, $|E|$ veces.

Según la estructura elegida para la cola de prioridad, obtenemos diferentes cotas temporales:

- **Array** Las operaciones de inserción y actualización de la clave toman un tiempo $\mathcal{O}(1)$, pero buscar y extraer el mínimo toma un tiempo $\mathcal{O}(|V|)$, ya que requiere recorrer todos los vértices. Así que el tiempo total de ejecución es $\mathcal{O}(|V|^2 + |E|) = \mathcal{O}(|V|^2)$.
- **Montículo binario (Binary min-heap):** Esta es la implementación eficiente estándar. Aquí, las operaciones de extracción del mínimo y actualización de la clave toman un tiempo $\mathcal{O}(\log |V|)$. Sumando las operaciones, el tiempo total de ejecución es $\mathcal{O}((|V| + |E|) \log |V|)$.

Al igual que en los algoritmos anteriores, la complejidad espacial es de $\mathcal{O}(|V|)$, ya que depende de la cola de prioridad utilizada.

Pseudocódigo

A continuación, se muestra un pseudocódigo del algoritmo Dijkstra:

Algoritmo 2: Dijkstra

```

Dijkstra(graph  $G$ , vertex  $A$ )
1: for  $v \in V(G) \setminus \{A\}$  do
2:    $parent[v] = \text{None}$ 
3:    $d[v] = \infty$ 
4: end-of-for
5:  $parent[A] = \text{None}$ 
6:  $d[A] = 0$ 
7:  $T = V(G)$ 
8:  $procesados = \emptyset$ 
9:  $u = A$ 
10: while  $T \neq \emptyset$  do
11:    $T = T \setminus \{u\}$ 
12:    $procesados = procesados \cup \{u\}$ 
13:    $N(u) = \text{getNeighborhood}(u)$ 
14:   for each  $v \in N(u)$  do
15:     if  $v \in T$  and  $d[v] > d[u] + w(u, v)$  then
16:        $d[v] = d[u] + w(u, v)$ 
17:        $parent[v] = u$ 
18:     end-of-if
19:   end-of-for
20:    $u = \text{GetMinEstimate}(T)$ 
21: end-of-while

```


Ejemplo de Dijkstra

Considérese ahora el siguiente grafo:

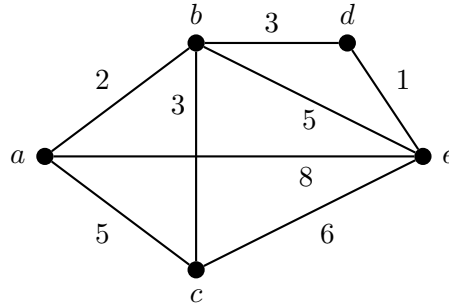


Figura 2.10: Grafo con pesos

Para el grafo 2.10 se tiene:

- $V(G) = \{a, b, c, d, e\}$
- $E(G) = \{(a, b), (a, c), (a, e), (b, c), (b, d), (b, e), (d, e), (c, e)\}$
- Los pesos de sus aristas: $w(a, b) = 2$, $w(a, c) = 5$, $w(a, e) = 8$, $w(b, c) = 3$, $w(b, d) = 3$, $w(b, e) = 5$, $w(c, e) = 6$, $w(d, e) = 1$

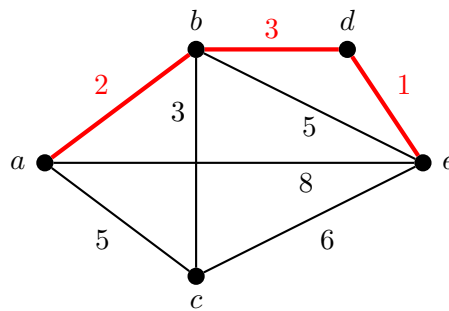
Se buscarán las rutas más cortas desde el vértice a al resto de vértices del grafo usando el algoritmo de Dijkstra. En la fase de inicialización se tiene $T = V(G) = \{a, b, c, d, e\}$ y, para los estimados y el arreglo *parent*:

- $d(a) = 0$, $d(b) = \infty$, $d(c) = \infty$, $d(d) = \infty$, $d(e) = \infty$
- $parent[a] = None$, $parent[b] = None$, $parent[c] = None$, $parent[d] = None$, $parent[e] = None$

Se muestra en la siguiente tabla la evolución de los estimados de los vértices, del arreglo *parent* y del conjunto de vértices no procesados T en cada iteración t del bucle while del algoritmo 2.5.3.

t	u	Vecinos v de $u \in T$	$d(v) > d(u) + w(u, v)$?	Estimados	Parent	T
1	a	b	$\infty > 0 + 2$ (V) $\Rightarrow d(b) = 2$	$d(a) = 0$	$parent[a] = \text{None}$	$\{b, c, d, e\}$
		c	$\infty > 0 + 5$ (V) $\Rightarrow d(c) = 5$	$d(b) = 2$	$parent[b] = a$	
		e	$\infty > 0 + 8$ (V) $\Rightarrow d(e) = 8$	$d(c) = 5$	$parent[c] = a$	
2	b	c	$5 > 2 + 3$ (F) No cambia	$d(d) = \infty$	$parent[d] = \text{None}$	$\{c, d, e\}$
		d	$\infty > 2 + 3$ (V) $\Rightarrow d(d) = 5$	$d(e) = 8$	$parent[e] = a$	
		e	$8 > 2 + 5$ (V) $\Rightarrow d(e) = 7$			
3	c	e	$7 > 5 + 6$ (F) No cambia	$d(a) = 0$	$parent[a] = \text{None}$	$\{d, e\}$
				$d(b) = 2$	$parent[b] = a$	
				$d(c) = 5$	$parent[c] = a$	
4	d	e	$7 > 5 + 1$ (V) $\Rightarrow d(e) = 6$	$d(d) = 5$	$parent[d] = b$	$\{e\}$
				$d(e) = 7$	$parent[e] = b$	
5	e	No hay		$d(a) = 0$	$parent[a] = \text{None}$	$\emptyset$
				$d(b) = 2$	$parent[b] = a$	
				$d(c) = 5$	$parent[c] = a$	
				$d(d) = 5$	$parent[d] = b$	
				$d(e) = 6$	$parent[e] = d$	

Tabla 2.4: Procesamiento de los vértices del grafo 2.10 en cada iteración de Dijkstra

Figura 2.11: Ruta más corta entre el vértice a y el vértice e

Corrección

Antes de la prueba de corrección, se enunciarán algunos lemas necesarios.

Corolario 2.4 (Propiedad de no existencia de camino). *Si no hay camino de u a v , entonces siempre se tiene $d[v] = \delta(u, v) = \infty$*

Lema 2.5 (Propiedad de convergencia). *Si $A \rightarrow u \rightarrow v$ es un camino más corto en G para unos $u, v \in V$, y si $d[u] = \delta(A, u)$ en cualquier momento anterior a relajar la arista (u, v) entonces $d[v] = \delta(A, v)$ en todo momento posterior.*

Lema 2.6 (Propiedad del límite superior). *Siempre tenemos $d[v] \geq \delta(A, v)$ para todos los vértices $v \in V$ y una vez $d[v]$ alcanza el valor $\delta(A, v)$, nunca cambia.*

Teorema 2.7. *Sea $G = (V, E)$ un grafo ponderado y no dirigido, con pesos no negativos. Supóngase que Dijkstra es ejecutado en G desde un vértice origen $A \in V$ dado, entonces termina con $d[u] = \delta(A, u)$ para todos los vértices $u \in V$.*

Demostración. La prueba se basa en demostrar la siguiente propiedad invariante:

“Al principio de cada iteración del bucle while (líneas 10-21), $d[v] = \delta(A, v)$ para cada vértice $v \in \text{procesados}$ ”.

Para esto es suficiente probar que para todo vértice $u \in V$ tenemos $d[u] = \delta(A, u)$ al añadir u a *procesados*.

A continuación, se prueba la propiedad invariante en las tres posibles fases:

- **Inicialización:** Antes de la primera iteración, tenemos $\text{procesados} = \emptyset$, por lo que, como no hay vértices evaluados, se tiene la propiedad.
- **Mantenimiento:** Queremos probar que para todo vértice $u \in V$ tenemos $d[u] = \delta(A, u)$ al añadir u a *procesados*.

Para llegar a una contradicción, sea u el primer vértice para el que $d[u] \neq \delta(A, u)$ cuando es añadido a *procesados* (es decir, que el camino encontrado por el algoritmo no es un camino más corto).

Como u es el primer vértice que no cumple la hipótesis, todos los vértices de *procesados* sí que la cumplen. Además, $u \neq A$, ya que A es el primer vértice añadido a *procesados* y $d[A] = \delta(A, A) = 0$ en ese momento.

Debe haber algún camino de A a u , ya que sino $d[u] = \delta(A, u) = \infty$ por la propiedad de no existencia de camino 2.4, lo que contradiría nuestra suposición de $d[u] \neq \delta(A, u)$.

Como hay al menos un camino, hay un camino más corto p desde A hasta u . Antes de añadir u a *procesados*, el camino p conecta un vértice en *procesados*, concretamente A , con un vértice en T , concretamente u . Consideremos el primer vértice y de p tal que $y \in T$, y sea $x \in \text{procesados}$ el predecesor inmediato a y en dicho camino p . Entonces, podemos descomponer el camino p como: $A \xrightarrow{p_1} x \rightarrow y \xrightarrow{p_2} u$ (los caminos p_1 o p_2 pueden no tener ninguna arista).

Se tiene que $d[y] = \delta(A, y)$ cuando u es añadido a *procesados*. Veámoslo. Primero, se tiene que $x \in \text{procesados}$. Entonces, como escogimos a u como el primer vértice para el que $d[u] \neq \delta(A, u)$ cuando es añadido a *procesados*, se tiene que $d[x] = \delta(A, x)$ cuando fue añadido a *procesados*. La arista (x, y) fue relajada en ese momento y, por la propiedad de convergencia 2.5 se tiene que $d[y] = \delta(A, y)$ cuando u es añadido a *procesados*.

Ahora se puede llegar a una contradicción para probar que $d[u] = \delta(A, u)$. Como y aparece antes de u en un camino más corto de A a u y todos los pesos son no negativos, se tiene $\delta(A, y) \leq \delta(A, u)$, y por tanto

$$d[y] = \delta(A, y) \leq \delta(A, u) \leq d[u] \text{ (por la propiedad del límite superior 2.6).}$$

Pero como ambos vértices u e y estaban en T (no estaban procesados) cuando u fue elegido en la línea 20, se tiene $d[u] \leq d[y]$. Por lo que las dos desigualdades anteriores son en realidad igualdades:

$$d[y] = \delta(A, y) = \delta(A, u) = d[u]$$

Por lo que $\delta(A, u) = d[u]$, lo que contradice a nuestra elección de u . Concluimos que $\delta(A, u) = d[u]$ cuando es añadido a *procesados*, y esta igualdad se mantiene en todo momento desde entonces.

- **Finalización:** Al terminar, $T = \emptyset$. Esto significa que *procesados* = V , y por lo tanto $d[u] = \delta(A, u)$ para todos los vértices del grafo.

□

2.6— Algoritmos de búsqueda informada

Los algoritmos de búsqueda informada o heurística son estrategias de inteligencia artificial en las que se utiliza un conocimiento específico más allá de la definición del problema para encontrar soluciones de forma más eficiente. Se seguirá en esta sección a [8].

2.6.1. Algoritmo A*

El algoritmo de búsqueda heurística por excelencia es el algoritmo A*. En él, se usa una función de evaluación $f(n) = g(n) + h(n)$. Las dos partes en las que se divide esta función de evaluación son:

- $g(n)$, que es el coste del camino desde el vértice de inicio al vértice n .
- $h(n)$ (**función heurística**), que es el coste estimado del camino más barato del vértice n al vértice destino.

Así que $f(n)$ es el coste estimado de la solución más barata a través de n .

Notemos que la función g equivale al estimado de la ruta más corta $d[v]$ que se presentó en el algoritmo de Dijkstra. De hecho si $h(v) = 0$, $\forall v \in V$ el algoritmo A* se reduce al algoritmo de Dijkstra.

Para que el algoritmo A* sea óptimo, es decir, que siempre encuentre la mejor solución posible, se necesita que la heurística $h(n)$ sea admisible.

Definición 2.29. Una heurística **admisible** es aquella que nunca sobreestime el coste de alcanzar el objetivo.

Como $g(n)$ es el coste real del camino desde el vértice de inicio al vértice n , se garantiza así que $f(n)$ nunca sobreestimaré el coste verdadero de una solución que pase por dicho vértice.

Sin embargo, para la aplicación de A^* a la búsqueda en grafos, se requiere una condición ligeramente más fuerte llamada consistencia.

Definición 2.30. Una heurística $h(n)$ es **consistente** si, para cada vértice n y cada sucesor n' de n generado por cualquier acción a , el coste estimado de alcanzar el objetivo desde n no es mayor que el coste del paso para llegar a n' más el coste estimado de alcanzar el objetivo desde n' :

$$h(n) \leq c(n, a, n') + h(n')$$

Esta condición es una forma de la desigualdad triangular. Toda heurística consistente es también admisible. Al utilizar una heurística consistente en una búsqueda sobre grafos, A^* garantiza la optimalidad de la solución encontrada.

El procedimiento algorítmico es similar al de Dijkstra, con dos diferencias fundamentales. Por un lado, la cola de prioridad se ordena en función del valor $f(n)$ en lugar de $g(n)$. Por otro lado, a diferencia de Dijkstra (donde la cola de prioridad es una optimización sobre la implementación con array), en A^* el uso de una cola de prioridad forma parte del diseño del algoritmo, ya que el orden de exploración depende del valor $f(n)$ actualizado dinámicamente. Por este motivo, se presenta directamente la versión con cola de prioridad mínima. El procedimiento se describe de la siguiente manera:

- Se asigna a cada nodo del grafo un coste real acumulado g , donde inicialmente $g[A] = 0$ para el vértice de origen y $g[v] = \infty$ para el resto de vértices. Se inicializa un registro de predecesores $parent$, donde $parent[v] = None$.
- Se inicializa la cola de prioridad mínima T , en la que se inserta inicialmente el vértice de origen A asociado a su coste total estimado $f[A] = g[A] + h(A) = 0 + h(A)$.
- Se inicializa un conjunto $procesados = \emptyset$ para llevar el control estricto de los vértices cuyo camino mínimo ya ha sido garantizado.
- Mientras la cola de prioridad no esté vacía ($T \neq \emptyset$):
 - Se extrae de T el vértice u que posea el menor valor $f(u)$ en ese instante.
 - Si $u \in procesados$, se ignora y se pasa a la siguiente iteración. En caso contrario, se añade al conjunto $procesados$. Si el vértice u extraído coincide con el vértice destino, se detiene la exploración (condición de parada).
 - Se recorren todos los vértices adyacentes a u que todavía no pertenezcan al conjunto $procesados$. Para cada uno de estos vecinos v , se comprueba la condición de relajación sobre el coste real g :

- * Si $g[v] > g[u] + w(u, v)$, se actualiza el coste mínimo $g[v] = g[u] + w(u, v)$ y se registra a u como su predecesor ($parent[v] = u$). Seguidamente, se calcula su nuevo coste total estimado $f[v] = g[v] + h(v)$ y se inserta el vértice v en la cola de prioridad T junto a este nuevo valor $f[v]$.
- * En caso contrario, se mantiene el valor actual de $g[v]$ y de $parent[v]$.

Al final del bucle, la reconstrucción del camino se realiza de la misma forma iterativa que en el algoritmo de Dijkstra, utilizando el diccionario *parent*.

Complejidad

La complejidad de A^* depende de la calidad de la función heurística empleada, del factor de ramificación y de la profundidad de la solución.

En el peor de los casos (por ejemplo, con una heurística nula $h(n) = 0$), A^* explora el grafo de la misma forma que Dijkstra. Mientras que con una heurística perfecta, el algoritmo recorrería únicamente los vértices que componen el camino óptimo, logrando un tiempo lineal. Por lo que no es posible responder en general cuál es la complejidad temporal de A^* .

Por otra parte, se tiene que el inconveniente principal de A^* no es su tiempo de ejecución, sino su consumo de memoria. Como mantiene todos los vértices generados en memoria dentro de las estructuras de control (la cola de prioridad y el conjunto de procesados), su complejidad espacial en el peor de los casos es exponencial.

Pseudocódigo

A continuación se muestra un pseudocódigo del algoritmo A*:

Algoritmo 3: A*

```

A*(graph  $G$ , vertex  $A$ , vertex  $t$ , heuristic  $h$ )
1: for  $v \in V(G) \setminus \{A\}$  do
2:    $parent[v] = \text{None}$ 
3:    $g[v] = \infty$ 
4: end-of-for
5:  $g[A] = 0$ 
6:  $T = \text{MinPriorityQueue}()$ 
7:  $\text{Insert}(T, (h(A), A))$ 
8:  $\text{procesados} = \emptyset$ 
9: while  $T \neq \emptyset$  do
10:   $(-, u) = \text{ExtractMin}(T)$ 
11:  if  $u \in \text{procesados}$  then
12:    continue
13:  end-of-if
14:   $\text{procesados} = \text{procesados} \cup \{u\}$ 
15:  if  $u == t$  then
16:    break
17:  end-of-if
18:   $N(u) = \text{getNeighborhood}(u)$ 
19:  for each  $v \in N(u)$  do
20:    if  $v \notin \text{procesados}$  and  $g[v] > g[u] + w(u, v)$  then
21:       $g[v] = g[u] + w(u, v)$ 
22:       $parent[v] = u$ 
23:       $f[v] = g[v] + h(v)$ 
24:       $\text{Insert}(T, (f[v], v))$ 
25:    end-of-if
26:  end-of-for
27: end-of-while

```

Ejemplo de A*

Considérese de nuevo el grafo 2.10 al que se le aplicará ahora el algoritmo de A*. A diferencia del ejemplo de Dijkstra, en el que se calcularon las rutas más cortas desde a hasta todos los vértices del grafo, A* requiere especificar un vértice destino, ya que la heurística estima el coste restante hasta dicho destino. Por tanto, se aplicará A* para calcular la ruta más corta desde el vértice a hasta el vértice e . Se usará la siguiente función heurística:

Vértice	a	b	c	d	e
$h(v)$	5	3	5	1	0

Tabla 2.5: Función heurística para el grafo 2.10

Esta heurística es admisible, ya que para todo vértice v se cumple $h(v) \leq \delta(v, e)$, donde $\delta(v, e)$ es el coste del camino más corto desde v hasta e .

En la fase de inicialización se tiene que T únicamente contiene al vértice a con su coste estimado $f(a) = g(a) + h(a) = 0 + 5 = 5$ y, además:

- $g(a) = 0, g(b) = \infty, g(c) = \infty, g(d) = \infty, g(e) = \infty$
- $parent[a] = None, parent[b] = None, parent[c] = None, parent[d] = None, parent[e] = None$

En la siguiente tabla se muestra la evolución de los costes reales acumulados g , los valores f , el arreglo $parent$ y el conjunto $procesados$ en cada iteración del bucle while del algoritmo 2.6.1.

t	u	Vecinos v no procesados	$g(v) > g(u) + w(u, v)$?	Estimados	Parent	Procesados
1	a	b	$\infty > 0 + 2$ (V) $\Rightarrow g(b) = 2, f(b) = 5$	$g(a) = 0$	$parent(a) = None$	{a}
		c	$\infty > 0 + 5$ (V) $\Rightarrow g(c) = 5, f(c) = 10$	$g(b) = 2$	$parent(b) = a$	
		e	$\infty > 0 + 8$ (V) $\Rightarrow g(e) = 8, f(e) = 8$	$g(c) = 5$	$parent(c) = a$	
2	b	c	$5 > 2 + 3$ (F) No cambia	$g(d) = \infty$	$parent(d) = None$	{a, b}
		d	$\infty > 2 + 3$ (V) $\Rightarrow g(d) = 5, f(d) = 6$	$g(e) = 8$	$parent(e) = a$	
		e	$8 > 2 + 5$ (V) $\Rightarrow g(e) = 7, f(e) = 7$	$g(a) = 0$	$parent(a) = None$	
3	d	e	$7 > 5 + 1$ (V) $\Rightarrow g(e) = 6, f(e) = 6$	$g(b) = 2$	$parent(b) = a$	{a, b, d}
				$g(c) = 5$	$parent(c) = a$	
				$g(d) = 5$	$parent(d) = b$	
4	e	No hay	$u = e$, parar	$g(e) = 6$	$parent(e) = d$	{a, b, d, e}
				$g(a) = 0$	$parent(a) = None$	
				$g(b) = 2$	$parent(b) = a$	

Tabla 2.6: Procesamiento de los vértices del grafo 2.10 en cada iteración de A*

Por tanto, A* encuentra la misma ruta más corta desde a hasta e que Dijkstra, pero A* ha procesado solo 4 vértices, mientras que Dijkstra procesó 5.

Corrección

Para la prueba de corrección se seguirá [4]. Antes de comenzar con la prueba de corrección, se enunciarán algunos lemas necesarios.

Lema 2.8. *Para cualquier vértice $v \notin \text{procesados}$ y para cualquier camino más corto P de A a v , existe un vértice $v' \in T$ en P tal que $g(v') = \delta(A, v')$.*

Corolario 2.9. *Supongamos $h(v) \leq \delta(v, t)$, para todo $v \in V$, y supongamos que A^* no ha terminado. Entonces, para cualquier camino más corto P desde A hasta t , existe un nodo $v' \in P$ con $f(v') \leq \delta(A, t)$.*

Teorema 2.10. *Sea $G = (V, E)$ un grafo ponderado y no dirigido, con pesos no negativos, y h una función heurística sobre los vértices de G . Si $h(v) \leq \delta(v, t)$ para todo $v \in V$, entonces A^* es admisible, es decir, A^* encuentra un camino más corto entre A y t .*

Demostración. El teorema se demostrará por reducción al absurdo. Supóngase que A^* no termina encontrando un camino óptimo entre A y el vértice destino t . Hay tres casos a considerar: el algoritmo termina en un vértice distinto a t , el algoritmo no termina o termina en el vértice destino t , pero sin alcanzar el coste mínimo.

- Caso 1: El algoritmo termina en un vértice distinto a t . Este caso contradice la condición de terminación/parada del algoritmo, por lo que lo descartamos.
- Caso 2: El algoritmo no termina. Supóngase que el coste de cualquier arista está acotado inferiormente por una constante positiva $\epsilon > 0$. Entonces, para cada vértice v que esté a más de $M = \frac{\delta(A, t)}{\epsilon}$ pasos de A , se tiene $f(v) \geq g(v) \geq \delta(A, v) \geq M\epsilon = \delta(A, t)$. Claramente, ningún vértice v a más de M pasos desde A puede ser extraído de T antes que t , pues por el corolario 2.9, habría algún vértice $v' \in T$ en un camino más corto tal que $f(v') \leq \delta(A, t) < f(v)$, por lo que A^* elegiría v' antes que v .

Entonces, sea $\chi(M)$ el conjunto de vértices accesibles en M o menos pasos desde A , el único motivo por el que A^* podría no terminar es que la cola de prioridad T nunca se vacíe dentro del conjunto $\chi(M)$. Sin embargo, siempre que un vértice $v \in \chi(M)$ es extraído de T y procesado, se añade a *procesados* y ya no se vuelve a procesar. Como $\chi(M)$ es finito y cada vértice solo puede ser añadido a *procesados* una vez, el número total de extracciones está acotado. Por tanto, A^* debe terminar en un número finito de pasos.

- Caso 3: El algoritmo termina en el vértice destino t , pero sin alcanzar el coste mínimo. Supóngase que A^* termina pero con $f(t) = g(t) > \delta(A, t)$. Por el corolario 2.9, existía, justo antes de la parada, un vértice $v' \in T$ en un camino más corto con $f(v') \leq \delta(A, t) < f(t)$. Por lo que en este paso, v' habría sido seleccionado en vez de t , contradiciendo que A^* ha terminado.

Como se han descartado todos los casos, se concluye que A^* es admisible. $\square$

Creación del grafo

En este capítulo, se describe el proceso de construcción del grafo que modela las relaciones entre canciones. El proceso se divide en dos fases: primero, el preprocesamiento del dataset, en el que se seleccionan y estandarizan los atributos más relevantes, y segundo, la creación del grafo, en la que se definen los vértices, las aristas y sus pesos mediante el algoritmo k -NN y ciertas reglas de compatibilidad entre géneros.

3.1— Librerías y módulos utilizados

La construcción del grafo se ha realizado en Python, apoyándose en una serie de librerías que se enumeran y explican brevemente a continuación:

- **pandas**: librería diseñada para el análisis y manipulación de datos. Se utiliza para cargar archivos .csv en un DataFrame, eliminar filas duplicadas y con valores nulos, reiniciar los índices del DataFrame tras la limpieza, guardar los datos procesados en un .csv y acceder a ciertas filas y columnas.
- **scikit-learn**: una de las librerías de aprendizaje automático más utilizadas. Se utilizan dos componentes de esta librería: con **StandardScaler** se estandarizan las características para que tengan media 0 y desviación típica 1, y con algunas herramientas de **NearestNeighbors** se implementa el algoritmo de k -NN.
- **NetworkX**: librería para el estudio y trabajo con grafos. Se utiliza para crear el grafo, añadirle vértices y aristas y obtener ciertas propiedades estructurales (verificar si es conexo, obtener el número de vértices y aristas y el grado de cada vértice, etc.).
- **sys** y **time**: módulos de la biblioteca estándar de Python. El primero se utiliza para finalizar el programa en caso de que no exista el archivo de entrada o el grafo no cumpla ciertas condiciones, mientras que el segundo se utiliza para medir el tiempo del proceso de construcción del grafo.

- **Matplotlib** y **seaborn**: librerías para la visualización de datos. Se usan para generar las gráficas del análisis de grados de los vértices y de pesos de las aristas del grafo.

3.2– Descripción y preprocesamiento del dataset

Para la realización de este trabajo se ha usado la siguiente base de datos [6], compuesta por 32834 canciones y sus respectivas características. Cada canción no solo está caracterizada por su título y artista, sino también por su género (y subgénero) musical y ciertos atributos numéricos obtenidos de la API de Spotify.

En concreto, se clasifican las canciones del conjunto de datos en los siguientes géneros: EDM (música electrónica, techno), rock, rap, pop, R&B (Rhythm and Blues) y latin (reggaetón, salsa, bachata).

En una primera versión, se escogió una base de datos que no proporcionaba el género de las canciones. Sin embargo, más adelante se explicará por qué esto no era viable.

Antes de seleccionar las características que definirán las distancias entre canciones, se aplican dos operaciones de limpieza sobre el dataset. En primer lugar, se eliminan las filas en las que el nombre de la canción y el del artista coinciden con los de otra entrada, conservando únicamente la primera ocurrencia. En segundo lugar, se descartan también las canciones con valores faltantes en alguno de los dos campos, ya que ambos sirven como identificador de cada canción del grafo.

Para calcular la similitud entre canciones y relacionarlas así de forma coherente es necesario que cada canción tenga un vector con las características más relevantes. Algunas de las características de la base de datos no aportan información útil para calcular la similitud entre canciones. Por ejemplo, la duración de las canciones o el hecho de que una canción esté grabada en vivo. Las características seleccionadas para la creación de las aristas son:

- **danceability**: Describe en qué medida es adecuada una canción para bailar basado en una combinación de elementos musicales incluyendo tempo, estabilidad rítmica, fuerza del compás y regularidad general. Un valor de 0.0 es lo menos bailable y 1.0 es lo más bailable.
- **energy**: Es una medida del 0.0 al 1.0 que representa la intensidad y actividad. Normalmente, una canción energética se siente rápida, potente y ruidosa. Por ejemplo, el heavy metal tiene una energía alta, mientras que la música clásica tiene una energía baja.
- **loudness**: Se mide en decibelios (dB). El valor del volumen se calcula como la media del volumen de toda la canción y es útil para comparar el volumen relativo de las canciones.
- **speechiness**: Detecta la presencia de palabras habladas en una canción. Cuanto más se parezca una grabación a un discurso, más cerca estará este valor a 1.

- **acousticness**: Una medida del 0.0 al 1.0 sobre si la canción es acústica. 1.0 indica un alto grado de confianza de que la canción es acústica.
- **instrumentalness**: Predice si una canción no contiene voces. Los sonidos “ooh” y “aah” son tratados como instrumental en este contexto. Las canciones de rap o habladas son claramente “vocales”. Cuanto más cerca esté este valor a 1.0, más probabilidad tendrá la canción de no contener contenido vocal. Los valores superiores a 0.5 se consideran canciones instrumentales, pero la fiabilidad aumenta cuanto más cerca esté este valor a 1.0.
- **valence**: Una medida del 0.0 al 1.0 que describe la positividad musical transmitida por la canción. Las canciones con un valor alto de “valence” suenan más positivas (felices, eufóricas), mientras que las canciones con un valor bajo suenan más negativas (triste, depresivas, enfadadas).
- **tempo**: El tempo estimado de una canción en pulsaciones por minuto (BPM). En terminología musical, el tempo es la velocidad o el ritmo de una pieza determinada y deriva directamente de la duración media de cada pulsación.

Aunque no se han incluido los atributos **key** y **mode** en el cálculo de la distancia actual, se han mantenido en el conjunto de datos procesado. Esta decisión se basa en su utilidad para posibles mejoras a la hora de perfeccionar las futuras transiciones.

Como se ha podido observar, las características presentan rangos muy diversos. Algunos atributos, como **energy**, oscilan entre 0 y 1, mientras que, por ejemplo, el **tempo** se mide en BPM y toma valores como 100, 150, 200. Así que, para que ninguna variable opaque a otra, resulta imprescindible escalar los datos.

Para ello, se ha aplicado una técnica de estandarización mediante la herramienta **StandardScaler** de la librería **scikit-learn** [9]. Este proceso transforma los valores de las variables continuas para que presenten una media de 0 y una desviación estándar de 1, garantizando así que ninguna variable domine sobre otra en el cálculo de las distancias.

3.3— Modelado del grafo

Una vez procesado el dataset, y definidos y estandarizados los atributos de cada canción, es necesario transformar este conjunto de datos en un modelo relacional estructurado. Para ello, se ha modelado el problema utilizando la teoría de grafos. Se ha definido un grafo $G = (V, E)$ no dirigido y ponderado, donde:

- V (Vértices): Representa el conjunto de canciones de la base de datos.
- E (Aristas): Representa las transiciones permitidas entre canciones, donde el peso $w(u, v)$ cuantifica el coste de la transición desde el vértice u al vértice v .

3.3.1. Elección de la métrica usada

Para evaluar la similitud entre dos canciones del espacio es fundamental elegir una métrica adecuada que represente fielmente la percepción musical del oyente. **En este trabajo, se ha optado por utilizar la distancia euclídea como métrica.** Para empezar, esta métrica presenta un bajo coste computacional, lo que resulta relevante al trabajar con un conjunto de datos de gran tamaño. Además, en la fase del preprocesamiento ya se han estandarizado las variables, por lo que todas las dimensiones contribuyen de forma equilibrada al cálculo de la distancia. Finalmente, esta métrica ofrece una interpretación geométrica directa, donde la similitud entre canciones se traduce en proximidad en el espacio de características. **Antes de tomar esta decisión,** se ha planteado el uso de otras medidas de similitud, que se han descartado por diversos motivos que se detallan a continuación.

Primero, se ha descartado el uso de la similitud del coseno, ya que ésta solo considera el ángulo entre dos vectores y no su magnitud. Por lo que dos canciones con proporciones similares pueden ser consideradas cercanas, incluso si los valores de sus atributos son opuestos. Por ejemplo, una canción de R&B (valores bajos de energy y loudness) y una de hard rock (valores muy altos de energy y loudness) podrían ser consideradas idénticas con la similitud del coseno, ya que sus vectores apuntan en la misma dirección, creando transiciones musicales poco naturales.

Otra opción que se planteó fue la distancia de Mahalanobis. Ésta fue descartada porque, al usar la matriz de covarianza, reduce la importancia de las variables correlacionadas, asumiendo que aportan información redundante. Sin embargo, en un espacio de características musicales, correlaciones en atributos como energy y loudness no son redundancias estadísticas, sino que son señales coherentes del carácter sonoro de una canción. Si se eliminasen las correlaciones, se distorsionaría la noción de similitud musical que se pretende capturar.

Finalmente, la taxi-distancia es menos sensible a los valores atípicos (outliers) que la distancia euclídea. Sin embargo, esta ventaja no es relevante en este trabajo: seis de los ocho atributos seleccionados (danceability, energy, speechiness, acousticness, instrumentality y valence) toman valores en $[0, 1]$ por definición de la API de Spotify, y los dos atributos restantes (loudness y tempo) están acotados de forma natural. Como todas las características están acotadas, no presentan outliers extremos que puedan distorsionar la distancia euclídea ni la estandarización. Por tanto, la taxi-distancia no aporta ventajas significativas frente a la euclídea.

En conclusión, se ha seleccionado la distancia euclídea para la construcción de las aristas y la asignación de pesos. Esta métrica combina eficiencia computacional con precisión en la representación, además de no eliminar las correlaciones entre características musicales, manteniéndolas como señales de similitud.

3.3.2. Compatibilidad de géneros

Como se mencionó antes, para la construcción de las aristas es importante tener en cuenta el género de las canciones, no solo la similitud matemática entre ellas. En una primera versión, esto no se tuvo en cuenta, lo que provocó el siguiente problema: si dos canciones pertenecientes a géneros opuestos (por ejemplo, latin y rock) compartiesen casualmente valores similares, podría crearse una conexión entre ellas, generando así una transición poco natural. Para que el sistema siga esta lógica y evite este problema, se han diseñado dos estructuras (matriz de afinidad y matriz de incompatibilidad) que indican la viabilidad de una transición entre un género y otro, clasificando las transiciones en tres categorías:

- **Admisible (Transiciones naturales):** En esta categoría están canciones que pertenecen al mismo género o aquellas que presentan una gran afinidad (por ejemplo, la transición del pop al EDM). En este caso, se crea la conexión manteniendo el peso de la arista igual a la distancia euclídea original.
- **Prohibido (Transiciones bruscas):** En esta categoría se encuentran canciones que pertenecen a géneros incompatibles (por ejemplo, la transición del rock a latin). En este caso, se rechaza la conexión.
- **Penalizado (Transiciones intermedias):** En esta categoría se encuentran aquellas canciones que no están explícitamente prohibidas, pero tampoco se consideran naturales. Como son transiciones viables pero no ideales, se aplica una penalización (Multiplicar por 2 la distancia) por crear esta conexión. De esta forma, solo se usarán estas rutas si no existe una alternativa mejor.

Las relaciones entre géneros se resumen en la siguiente tabla:

Género	EDM	ROCK	RAP	POP	R&B	LATIN
EDM	Admisible	Penalizado	Penalizado	Admisible	Penalizado	Penalizado
ROCK	Penalizado	Admisible	Prohibido	Admisible	Penalizado	Prohibido
RAP	Penalizado	Prohibido	Admisible	Penalizado	Admisible	Admisible
POP	Admisible	Admisible	Penalizado	Admisible	Admisible	Admisible
R&B	Penalizado	Penalizado	Admisible	Admisible	Admisible	Admisible
LATIN	Penalizado	Prohibido	Admisible	Admisible	Admisible	Admisible

Tabla 3.1: Tabla con las restricciones de género

En una segunda versión, la categoría de transiciones intermedias no existía, por lo que el sistema resultaba demasiado estricto, lo que empeoraba también la calidad de las transiciones.

Estas matrices nos ayudan en la construcción del grafo, asegurando que las conexiones entre canciones sean musicalmente coherentes.

3.3.3. Construcción de las aristas mediante k -NN

En un modelo donde fuese posible transicionar desde una canción a cualquier otra, obtendríamos un grafo completo de $\frac{|V|(|V|-1)}{2}$ aristas. Computacionalmente, esto resultaría muy ineficiente e innecesario, ya que se crearían conexiones entre canciones acústicamente incompatibles.

Para solucionar este problema, se ha optado por crear un grafo disperso usando el algoritmo de los k vecinos más cercanos (k -NN). Gracias a este algoritmo, se pueden establecer aristas únicamente entre aquellas canciones que se encuentran próximas en el espacio definido por las ocho características mencionadas anteriormente.

Para obtener un grafo con la densidad adecuada es esencial realizar una correcta elección del parámetro k : un valor excesivamente grande generaría conexiones innecesarias, mientras que un valor demasiado pequeño podría desconectar el grafo. Se definen dos parámetros clave:

- $k_{objetivo} = 11$: Representa el número de transiciones que se desea tener para cada canción. Se define $k_{objetivo} = 11$ (y no 10) para compensar el descarte de la conexión de una canción consigo misma.
- $k_{busqueda} = 4 \cdot k_{objetivo}$: Se configura el algoritmo para extraer los 44 vecinos más cercanos, de los que se descarta la propia canción, quedando así 43 candidatos. Este valor permite tener candidatos reservados en caso de que se descarten los vecinos más cercanos por las reglas de género.

Una vez definidos estos parámetros, se procede a explicar el proceso de creación de aristas:

Para cada vértice, el algoritmo recorre la lista de candidatos propuestos por k -NN y se consulta el tipo de relación que tienen los géneros. Si la conexión es prohibida se ignora. En caso contrario, se registra la arista con su peso correspondiente (original o con penalización). Una vez se ha alcanzado el $k_{objetivo}$, se detiene el proceso para ese vértice y se continúa con el siguiente.

Si hubiese alguna canción que no tuviera ninguna conexión tras evaluar todos los candidatos propuestos (por incompatibilidades de género), el algoritmo ignora de forma excepcional la matriz de incompatibilidad y la conecta con su vecino más cercano. A esta arista se le aplica una penalización grave (Multiplicar por 10 la distancia), para asegurar que solo se use en situaciones de necesidad extrema, evitando así el aislamiento del vértice.

Es fundamental recordar que la relación de vecindad en k -NN es inherentemente asimétrica (el hecho de que u considere a v su vecino no garantiza que v elija a u). Como se requería un grafo no dirigido para los algoritmos de búsqueda, se ha utilizado la clase `Graph` de `NetworkX`, que representa grafos no dirigidos, de forma que cada arista añadida es accesible desde ambos extremos. Por lo tanto, se establece una arista bidireccional definitiva si alguna de las dos canciones selecciona a la otra como vecina.

Teniendo esto en cuenta, aunque un vértice solo genere sus aristas de salida, puede acabar aumentándose su grado de conectividad al recibir conexiones de otros vértices que lo han considerado como vecino.

Por último, dadas las restricciones impuestas por las relaciones entre géneros, es posible que ciertas canciones queden aisladas. Para garantizar que no haya demasiadas canciones aisladas, se realiza un análisis de conectividad. Tras construir el grafo, se verifica si este es conexo. En caso contrario, se extrae la componente conexas más grande. Si esta abarca más del 90 % del dataset, las canciones desconectadas se descartan y se toma dicho subgrafo como grafo final.

3.4— Representación computacional

Una vez construido el grafo $G = (V, E)$, es necesario elegir una estructura de datos adecuada para almacenarlo. Como se explicó en la Sección 2.3, la elección entre matriz y lista de adyacencia depende de la densidad del grafo.

En este caso, se tiene un grafo con $|V| \approx 30000$ vértices y un grado medio de aproximadamente 14 aristas por vértice. Por lo que tenemos un grafo disperso ya que el orden de las aristas es de $\mathcal{O}(|V|)$, muy inferior al máximo de $\mathcal{O}(|V|^2)$.

Una matriz de adyacencia requeriría almacenar $|V|^2 \approx 9 \times 10^8$ entradas. Como la mayoría de entradas estarían vacías, esto supondría un desperdicio de memoria y un excesivo coste computacional. Por este motivo, se ha optado por representar el grafo mediante una lista de adyacencia, que es mucho más eficiente en este caso.

Para ello, se ha utilizado la clase **Graph** de la librería **NetworkX**, que implementa internamente esta estructura. Cada nodo almacena un diccionario de sus vecinos, donde cada entrada contiene el peso de la arista correspondiente. Además, las aristas se mantienen en un archivo CSV, con lo que se puede cargar el grafo directamente en futuras ejecuciones sin necesidad de reconstruirlo.

3.5— Análisis del grafo final

De las 32834 canciones del conjunto de datos original, se seleccionaron finalmente 26229 canciones tras el preprocesamiento, donde se eliminaron canciones con título y artista duplicados, así como las filas con valores faltantes en alguno de estos campos.

Como se puede observar en la figura 3.1, hay una mayor concentración de vértices con un grado de conectividad entre las 10 y 15 conexiones. El grado medio es de 13.78, mayor que el $k_{objetivo} = 11$. Este resultado es coherente con el diseño de las conexiones explicado anteriormente: aunque un vértice genera, como máximo, 10 aristas de salida, puede recibir conexiones de otros vértices que lo han elegido como vecino, aumentando así su grado final. Además, el grado mínimo es 4 y el grado máximo es 30, confirmando que no hay canciones aisladas y tampoco canciones que concentren demasiadas conexiones.

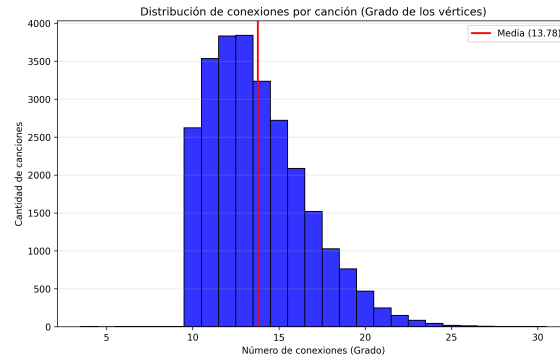


Figura 3.1: Gráfica grados

Por otra parte, con respecto al coste de las transiciones, en la figura 3.2 se puede observar que la distribución de los pesos está concentrada entre 0 y 2, con una media de 1.0067. Hay 143789 conexiones naturales (sin penalización), 36890 conexiones con penalización moderada y ninguna con penalización severa. Esto confirma que el sistema que se ha modelado funciona según lo esperado: se han descartado las transiciones bruscas, priorizado las transiciones naturales entre géneros afines y las transiciones con penalización moderada han servido como puente entre distintos géneros cuando no existía una alternativa mejor.

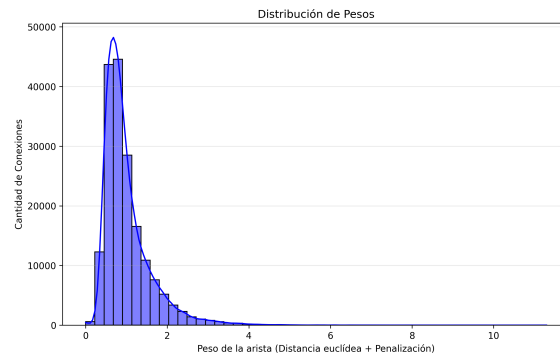


Figura 3.2: Gráfica pesos

Implementación de los algoritmos de búsqueda

En este capítulo, se describen las implementaciones en Python de los algoritmos de BFS, Dijkstra y A*, adaptados al objetivo del trabajo: encontrar una secuencia de canciones que conecte una canción de origen con una canción de destino.

4.1— Librerías y módulos utilizados

Para la implementación de los algoritmos de búsqueda se han empleado algunas librerías y módulos de Python, intentando mantener el estilo de los pseudocódigos descritos en el Capítulo 2, aunque realizando ciertos cambios para mejorar la eficiencia.

- **NetworkX**: librería de Python para el trabajo con grafos. Se utiliza para obtener los vértices y vecinos de un vértice dado y para acceder a los pesos de ciertas aristas.
- **collections.deque**: estructura de datos de tipo cola del módulo **collections** de la biblioteca estándar de Python. Se utiliza en la implementación de BFS para operaciones con la cola. En particular, se usan las operaciones **popleft** (desencolar al principio) y **append** (encolar al final), que tienen complejidad $\mathcal{O}(1)$, lo que permite mantener la complejidad $\mathcal{O}(|V| + |E|)$ del algoritmo.
- **heapq**: módulo de la biblioteca estándar de Python. Se utiliza en los algoritmos de Dijkstra y A* para implementar un montículo binario como cola de prioridad, ya que esto reduce la complejidad de Dijkstra, tal y como se explicó en la sección 2.5.3. En concreto, se usan las funciones **heapq.heappop** (extracción del mínimo) y **heapq.heappush** (inserción de un elemento).
- **math**: módulo de la biblioteca estándar de Python. Se utiliza la función **math.sqrt** para el cálculo de la heurística en A*.

4.2– Breadth First Search (BFS)

El algoritmo BFS, descrito en la Sección 2.5.2, sigue una exploración por capas y garantiza encontrar el camino con el menor número de aristas entre dos vértices. En el contexto de este trabajo, BFS se utiliza para encontrar la secuencia más corta de canciones que conecta una canción origen con una canción destino, ignorando el peso de las aristas. A continuación se presentan tres versiones del algoritmo: la primera reproduce el pseudocódigo del Capítulo 2 y las dos siguientes introducen diferentes optimizaciones.

4.2.1. Primera versión

En esta primera versión, se reproduce el pseudocódigo 2.5.2 del Capítulo 2, añadiendo dos modificaciones para adaptarlo al objetivo del trabajo. Primero, se introduce el parámetro `end_vertex`, para finalizar la búsqueda cuando se alcance el vértice destino, y segundo, se reconstruye el camino recorriendo el diccionario `parent` desde `end_vertex` hasta `start_vertex` e invirtiendo el resultado. Se ha incluido en esta versión la variable booleana `found` para saber si la búsqueda ha finalizado con éxito o si se ha vaciado la cola sin encontrar el destino. Esta versión mantiene las estructuras del pseudocódigo: el diccionario `status` (con los tres estados), la cola `Q` y el diccionario `parent`. Se omite el diccionario `d`, ya que solo se necesita reconstruir el camino y, para esto, basta con `parent`.

4.2.2. Segunda versión

Para mejorar la primera versión, se observa que, aunque el diccionario `status` mantiene los tres estados posibles (no procesado (1), descubierto (2) y procesado (3)), en realidad BFS realiza una única comprobación: si un vértice está en estado 1 (no descubierto), se encola. La distinción entre los estados 2 (descubierto, en cola) y 3 (procesado) no afecta a ninguna decisión del algoritmo, ya que en BFS cada vértice se encola exactamente una vez.

Nótese que con el diccionario `parent` se puede comprobar si un vértice ya ha sido descubierto, ya que a un vértice se le asigna entrada en `parent` justo cuando se descubre. Por tanto, la condición `status[j] == 1` es equivalente a `j not in parent`, y el diccionario `status` puede eliminarse.

```
1 for j in graph.neighbors(x): #recorremos los vecinos de x
2     if j not in parent:      #si el vertice j no ha sido descubierto
3         parent[j] = x
4         Q.append(j)          #encolamos el vertice j
```

Código 4.1: Comprobación de vértices descubiertos sin el diccionario `status`

Además, se elimina la variable `found` que permitía saber si se había llegado al vértice destino. En esta versión, la función devuelve el camino directamente cuando se llega al vértice destino y devuelve `None` si el bucle finaliza sin encontrarlo, sin necesidad de usar una variable intermedia.

4.2.3. Tercera versión

En esta última versión, se adelanta la comprobación del vértice destino al bucle `for`, justo cuando se descubre como vecino. De esta forma, la función devuelve el camino directamente, sin necesidad de encolar al vértice destino y al resto de vecinos, ni de continuar procesando el resto de vértices que estuvieran en la cola.

```
1 for j in graph.neighbors(x): #recorremos los vecinos de x
2     if j not in parent: #si el vertice j no ha sido descubierto
3         parent[j] = x
4
5         if j == end_vertex: #hemos llegado al vertice destino
6             camino = []
7             v_actual = j
8             while v_actual is not None:
9                 camino.append(v_actual)
10                v_actual = parent[v_actual]
11                return camino[::-1]
12
13     Q.append(j) #encolamos el vertice j
```

Código 4.2: Detección anticipada del destino dentro del bucle `for`

En la segunda versión, la comprobación se realizaba al desencolar el destino, por lo que desde que se descubría el vértice destino hasta que se desencolaba podrían transcurrir varias iteraciones del bucle `while`. En esta última versión, se evitan todas estas iteraciones, ya que la búsqueda finaliza en cuanto se encuentra al destino entre los vecinos de algún vértice extraído. Esta mejora supone un ahorro relevante, ya que en este trabajo los vértices tienen un grado medio de 14, por lo que cada iteración del bucle `while` supone revisar aproximadamente 14 vértices.

4.3— Dijkstra

A diferencia de BFS, que busca el camino con menor número de aristas, el algoritmo de Dijkstra tiene en cuenta los pesos de las aristas y su objetivo es encontrar el camino con menor coste desde un vértice origen a un vértice destino. En el contexto de este trabajo, Dijkstra se utiliza para encontrar la transición más suave entre una canción de origen y una canción de destino. A continuación, se presentan dos versiones del algoritmo: la primera reproduce el pseudocódigo del Capítulo 2 y la segunda introduce una optimización utilizando una cola de prioridad con la librería `heapq`.

4.3.1. Primera versión

En esta primera versión, se reproduce el pseudocódigo 2.5.3 del Capítulo 2, con dos modificaciones. Por un lado, se introduce el parámetro `end_vertex`, para finalizar la búsqueda

cuando se alcance el vértice destino. La reconstrucción del camino se realiza de forma análoga a BFS: se recorre el diccionario `parent` desde el vértice destino hasta el vértice origen y se invierte la secuencia obtenida. Por otro lado, se elimina el conjunto `procesados`, ya que en esta implementación la estructura `T` (que contiene los vértices no procesados) ya proporciona la información necesaria: un vértice está procesado si y solo si no pertenece a `T`.

La extracción del vértice con menor estimado se realiza mediante una función auxiliar llamada `estimado_minimo` (que se corresponde con `GetMinEstimate` en el pseudocódigo 2.5.3). Esta función recorre los elementos de `T` comparando los valores almacenados en el diccionario `d`, lo que supone un coste $O(|V|)$ por cada extracción y, por tanto, un coste total de $O(|V|^2)$, como se analizó en la Sección 2.5.3.

4.3.2. Segunda versión

En esta segunda versión, se van a utilizar las funciones `heapq.heappop` y `heapq.heappush` de la librería `heapq`. En lugar de usar un array, usaremos un binary min-heap (montículo binario), que es más eficiente. Como ya se señaló en el Capítulo 2, la complejidad pasa de ser $O(|V|^2)$ a $O((|V| + |E|) \log |V|)$.

En la versión con array, `T` es una lista que contiene una vez a cada vértice no procesado. Cuando relajamos la distancia de un vértice `v`, simplemente actualizamos ese valor. Por lo que `v` sigue estando en `T` solo una vez. Cuando por fin se extrae `v`, sale con su distancia óptima y se elimina de `T`. Por este motivo, solo hay que saber si `v` está o no en `T`, sin necesidad del conjunto `procesados`.

Sin embargo, en esta versión, el heap almacena pares `(distancia, v)`. Cuando relajamos la distancia de un vértice `v`, no es posible modificar su antiguo valor en el heap, ya que los heaps no permiten una actualización eficiente. En vez de esto, se inserta una entrada nueva `(distancia_mejorada, v)`. Por lo tanto, un vértice `v` puede estar varias veces en el heap `T` con distancias diferentes. Como un binary min-heap siempre extrae el vértice con menor valor, la primera entrada de `v` que se extraerá será la asociada a su distancia óptima. Las entradas con distancias mayores permanecen en el heap y serán extraídas en algún momento. Para evitar procesar un mismo vértice más de una vez, se reintroduce el conjunto `procesados`. Cada vez que se extrae un vértice del heap, se comprueba si pertenece a `procesados`. Si es así, se pasa a la siguiente iteración. Así cada vértice se procesa una sola vez con su distancia óptima.

```

1 while T:
2     dist, u = heapq.heappop(T)
3     if u in procesados:
4         continue
5     procesados.add(u)
6     ...

```

Código 4.3: Filtro para evitar procesar vértices repetidos

4.4– A*

Al igual que Dijkstra, el objetivo de A* es encontrar el camino con menor coste desde un vértice origen a un vértice destino. En el contexto de este trabajo, A* se utiliza para encontrar la transición más suave entre una canción de origen y una canción de destino. Como A* utiliza una función heurística para orientar la búsqueda, puede recorrer menos vértices que Dijkstra, lo que lo hace potencialmente más eficiente, dependiendo de la heurística empleada.

Para este algoritmo se presenta una única versión que reproduce el pseudocódigo 2.6.1 del Capítulo 2.

Lo más importante en este algoritmo es la elección de la función heurística. Se ha optado en este trabajo por la distancia euclídea entre los vectores de características de las canciones. A continuación, se explica por qué esta heurística es admisible en el grafo construido en el Capítulo 3, lo que garantiza que A* encontrará siempre el camino óptimo entre dos vértices cualesquiera.

4.4.1. Justificación de la heurística elegida

La heurística elegida es la distancia euclídea entre el vértice actual y el vértice destino: dada una canción destino $t \in V$, se define $h(v) = d_e(v, t)$ para todo $v \in V$. A continuación se demuestra que esta heurística es admisible sobre el grafo construido en el Capítulo 3, es decir, que $h(v) \leq \delta(v, t)$ para todo $v \in V$.

El peso de una arista (u, v) se define como $w(u, v) = d_e(u, v) \cdot p(u, v)$, donde $p(u, v) \in \{1, 2, 10\}$ es la penalización aplicada según la compatibilidad de géneros. Como las penalizaciones son siempre mayores o iguales a 1, se tiene que el peso de una arista es siempre mayor o igual a la distancia euclídea entre los vértices que la forman, es decir, $w(u, v) \geq d_e(u, v)$.

Sea $v \in V$ un vértice cualquiera y sea $P = (v_0 = v, v_1, \dots, v_n = t)$ un camino cualquiera de v a t . Se tiene que el peso total del camino es la suma de los pesos de sus aristas que, por el párrafo anterior, verifica

$$w(P) = \sum_{i=1}^n w(v_{i-1}, v_i) \geq \sum_{i=1}^n d_e(v_{i-1}, v_i).$$

Aplicando reiteradamente la desigualdad triangular de la distancia euclídea, se tiene que la suma de las distancias entre los segmentos del camino es mayor o igual que la distancia entre los extremos:

$$\sum_{i=1}^n d_e(v_{i-1}, v_i) \geq d_e(v_0, v_n) = d_e(v, t).$$

Así que $w(P) \geq d_e(v, t)$ y, como $h(v) = d_e(v, t)$, se tiene que: $w(P) \geq h(v)$, para todo camino P de v a t . Como esto se cumple para cualquier camino, en particular, se

cumple para el camino de coste mínimo: $\delta(v, t) \geq h(v)$. Por lo tanto, esta heurística es admisible y, en consecuencia, A* con esta heurística siempre encuentra el camino óptimo entre cualquier par de canciones del grafo construido en el Capítulo 3.

En el código, esta heurística se implementa en la función auxiliar `heuristica_euclidea` que, dadas dos canciones, calcula la distancia euclídea entre sus vectores de características.

Comparativas

En este capítulo se realiza una comparativa entre los diferentes algoritmos implementados en el capítulo anterior sobre el grafo de canciones construido en el Capítulo 3. Tras definir los criterios de comparación y seleccionar cuatro pares de canciones, se realizan las siguientes comparativas: BFS frente a Dijkstra (algoritmos con distintos objetivos) y Dijkstra frente a A* (mismo objetivo pero con diferente eficiencia). A continuación, se muestra una visión global del comportamiento de los tres algoritmos. Por último, se muestran los resultados de forma interactiva mediante una aplicación creada con la librería `Streamlit`.

5.1— Librerías y módulos utilizados

Para la realización de las comparativas y el desarrollo de la aplicación interactiva se han usado las siguientes librerías y módulos de Python. Salvo `Streamlit`, todos se han presentado en capítulos anteriores, por lo que aquí se indica únicamente el uso concreto que se les da en este capítulo.

- **time**: se utiliza la función `time.perf_counter()` para medir los tiempos de ejecución de cada algoritmo. Esta función es más precisa que `time.time()` y es más adecuada para comparar períodos breves de tiempo, como en este caso.
- **pandas**: se utiliza para cargar el grafo (la lista de aristas en formato `.csv`) y el conjunto de canciones con sus características.
- **NetworkX**: se utiliza para reconstruir el grafo y consultar las aristas y sus pesos durante la ejecución de los algoritmos.
- **Streamlit**: librería que permite crear interfaces a partir de código en Python, sin necesidad de programar en HTML, CSS o JavaScript. Ofrece componentes como desplegables y botones, y gestiona automáticamente el refresco de la interfaz.

5.2– Criterios de comparación

Para comparar los diferentes algoritmos, se han seleccionado las siguientes métricas:

- **Tiempo de cálculo:** Tiempo (medido en segundos) que cada algoritmo tarda en encontrar el camino entre la canción origen y la canción destino. Para medir el tiempo se ha utilizado `time.perf_counter()` de la librería `time`.
- **Vértices explorados:** Número de vértices procesados por el algoritmo durante la búsqueda del camino.
- **Coste de la ruta:** Suma de los pesos de las aristas que forman el camino encontrado.
- **Salto (Canciones):** Número de transiciones entre canciones que componen el camino, es decir, longitud del camino encontrado (número de aristas).

5.3– Casos de estudio

Para evaluar el comportamiento de los algoritmos en distintos escenarios, se han seleccionado cuatro pares de canciones de origen y destino que representan transiciones musicales de dificultad creciente. En este caso, la dificultad se mide mediante la distancia euclídea entre los vectores de características de las canciones de origen y destino. Cuanto mayor sea esta distancia, más complicado será encontrar una transición musical coherente entre ellas.

Hay dos motivos principales que justifican esta elección. Por un lado, esta distancia es independiente de los algoritmos y del grafo, por lo que constituye una caracterización objetiva del par (origen, destino). Por otro lado, coincide con la heurística elegida y, por la admisibilidad probada en la sección 4.3.1, supone una cota inferior del coste de cualquier camino entre la canción origen y la canción destino.

En la siguiente tabla se presentan los cuatro pares de canciones seleccionados con sus respectivos géneros, ordenados de menor a mayor dificultad.

nº	Canción origen	Canción destino	Distancia
1	Fix You - Coldplay (Pop)	Yellow - Coldplay (Pop)	2.63493
2	Mockingbird - Eminem (Rap)	Whatever It Takes - Imagine Dragons (Pop)	3.46890
3	Shape of You - Ed Sheeran (Pop)	Enter Sandman - Metallica (Rock)	4.05102
4	Me Gustas Tu - Manu Chao (Latin)	Knockin' On Heaven's Door - Guns N' Roses (Rock)	4.29205

Tabla 5.1: Pares seleccionados para el estudio, ordenados de menor a mayor distancia euclídea entre sus vectores de características

La transición 1 es la más sencilla. Las canciones pertenecen al mismo artista y al mismo género.

La transición 2 cruza desde el rap hasta el pop. Esta transición está clasificada como penalizada según las reglas establecidas en el Capítulo 3, lo que obligará a los algoritmos a buscar géneros intermedios o a aceptar las penalizaciones correspondientes.

Aunque en la transición 3 se tiene que los géneros (pop y rock) son compatibles según el modelo, las canciones presentan diferencias acústicas significativas (pop melódico frente al heavy metal), lo que se traduce en una gran distancia entre sus vectores de características.

Por último, la transición 4 es la más extrema. Los géneros latin y rock fueron marcados como incompatibles según las restricciones de género, por lo que no hay ninguna arista que conecte estas dos canciones y los algoritmos se verán obligados a construir el camino mediante aristas intermedias.

5.4– BFS frente a Dijkstra

La comparación entre estos dos algoritmos resulta interesante, ya que tienen objetivos distintos: BFS busca el camino con menor número de aristas, sin tener en cuenta los pesos, y Dijkstra busca el camino de coste mínimo. Por lo tanto, es de esperar que el coste de la ruta encontrada por BFS sea mayor o, como mucho, igual que el coste de la ruta encontrada por Dijkstra.

Como BFS no tiene en cuenta el peso de las aristas, es necesario calcular el peso de la ruta obtenida:

```
1 coste_bfs = sum(graph[u][v]['weight'] for u, v in zip(camino[:-1], camino[1:]))
```

Código 5.1: Cálculo del peso de la ruta obtenida con BFS

En el contexto de este trabajo, BFS dará la playlist con la transición más corta (menos canciones intermedias), mientras que Dijkstra dará la transición más suave (menor coste).

La siguiente tabla muestra el resultado obtenido al ejecutar ambos algoritmos con los cuatro pares de canciones seleccionados, con las métricas descritas en la sección anterior.

nº	Algoritmo	Tiempo (s)	Vértices explorados	Coste de la ruta	Salto (Canciones)
1	BFS	0.00	236	4.8810	4
	Dijkstra	0.01	296	3.4649	4
2	BFS	0.01	11633	7.4189	8
	Dijkstra	0.08	10854	5.8642	9
3	BFS	0.01	10282	10.6151	8
	Dijkstra	0.07	10499	6.0648	10
4	BFS	0.01	7417	8.2295	8
	Dijkstra	0.09	11352	7.5337	11

Tabla 5.2: Comparativa entre BFS y Dijkstra sobre los cuatro pares seleccionados.

Se estudia ahora la diferencia en las métricas en cada algoritmo.

Tiempo de cálculo. BFS resulta más rápido que Dijkstra en todos los casos, aunque la diferencia es mínima cuando las canciones de origen y destino se encuentran próximas, y aumenta conforme crece esta distancia. La rapidez de BFS se debe a que no tiene que extraer el mínimo de la cola de prioridad ni verificar la condición de relajación.

Vértices explorados. La relación entre los vértices explorados en ambos algoritmos no es uniforme. BFS detiene la exploración en cuanto cualquier camino alcanza al destino, mientras que Dijkstra debe seguir procesando vértices hasta que comprueba que ninguno mejora el camino obtenido. En los casos 1, 3 y 4, Dijkstra explora más vértices que BFS, mientras que en el caso 2 ocurre lo contrario. Esta variabilidad se debe a que la cuenta de los vértices explorados depende de magnitudes distintas en cada algoritmo. BFS explora todos los vértices que están a una distancia (en número de saltos) inferior a la del destino, mientras que Dijkstra explora todos los vértices cuyo coste acumulado no supera el mínimo encontrado. Como estas dos cantidades dependen de la estructura del grafo, su relación varía de un caso a otro.

Coste de la ruta y saltos. Como era de esperar, en todos los casos Dijkstra ha encontrado la ruta de menor coste y BFS la ruta con el menor número de saltos. La diferencia de coste varía entre casos, destacando el caso 3, donde el camino de BFS es más de un 75 % más caro que el de Dijkstra. En el caso 4, la diferencia de costes no es muy significativa. La diferencia en el número de saltos va en aumento conforme crece la dificultad. Dijkstra toma caminos más largos si ello supone evitar aristas penalizadas (en el caso 4, 11 saltos frente a 8 saltos de BFS).

Ahora, para estudiar la calidad musical de las transiciones, veamos las rutas obtenidas por ambos algoritmos en el caso 3, que es el que presenta mayor diferencia de costes.

Playlist BFS	Playlist Dijkstra
0. [POP] Shape of You - Ed Sheeran	0. [POP] Shape of You - Ed Sheeran
1. [R&B] On The Low - Burna Boy	1. [ROCK] You Sexy Thing - Hot Chocolate
2. [LATIN] Talento De Televisión - Willie Colón	2. [ROCK] This Guy's In Love With You Pare - Chito Miranda
3. [EDM] Instant Funk - Merchant	3. [ROCK] Since You Been Gone - RAINBOW
4. [ROCK] I Only Want to Be with You - Bay City Rollers	4. [POP] Sweet Dreams (Are Made of This) - Remastered - Eurythmics
5. [ROCK] China Grove - The Doobie Brothers	5. [ROCK] Double Vision - Foreigner
6. [ROCK] 25 or 6 to 4 - Chicago	6. [ROCK] Can't Get Enough (Remastered Album Version) - Bad Company
7. [ROCK] Two Tickets To Paradise - Eddie Money	7. [POP] Hitori no Yoru wo Nuke - Lucky Kilimanjaro
8. [ROCK] Enter Sandman - Metallica	8. [POP] When I'm Away - The Colourist
	9. [ROCK] Shot In The Dark - Ozzy Osbourne
	10. [ROCK] Enter Sandman - Metallica

Tabla 5.3: Playlists generadas por BFS y Dijkstra para el camino desde “Shape of You - Ed Sheeran” hasta “Enter Sandman - Metallica”

Se aprecia que hay una notable diferencia musical entre ambas playlists. La playlist de BFS atraviesa cinco géneros diferentes (pop, R&B, latin, EDM y rock) en las primeras cinco canciones, lo que resulta en cambios muy bruscos y un coste elevado. Sin embargo, la playlist de Dijkstra se mantiene en los géneros de pop y rock, que el modelo considera compatibles, generando transiciones más suaves, a costa de una playlist más larga. Esto valida el modelo de penalizaciones diseñado en el Capítulo 3. Las transiciones entre géneros no compatibles se penalizan lo suficiente como para que Dijkstra seleccione caminos más largos pero musicalmente más coherentes.

En conclusión, BFS es la mejor opción si lo que se busca es minimizar el número de saltos y el tiempo de ejecución. Sin embargo, si se quiere obtener una playlist con transiciones musicales coherentes, Dijkstra es el algoritmo ideal.

5.5– Dijkstra frente a A^*

A^* y Dijkstra resuelven el mismo problema: encontrar el camino óptimo entre dos vértices del grafo. Dijkstra explora los vértices sin tener en cuenta dónde se encuentra el destino. Sin embargo, A^* incorpora una heurística que estima el coste restante hasta llegar al destino y permite procesar en cada iteración el vértice más prometedor.

En la Sección 4.3.1 se demostró que la heurística elegida en este trabajo (distancia euclídea) es admisible, lo que garantiza que A* siempre encuentra el camino óptimo, al igual que Dijkstra. Por tanto, es de esperar que ambos algoritmos coincidan en el coste y en el número de saltos del camino encontrado, pero que A* explore menos vértices y emplee menos tiempo.

La siguiente tabla muestra el resultado obtenido al ejecutar ambos algoritmos con los cuatro pares de canciones seleccionados.

nº	Algoritmo	Tiempo (s)	Vértices explorados	Coste de la ruta	Salto (Canciones)
1	Dijkstra	0.01	296	3.4649	4
	A*	0.01	14	3.4649	4
2	Dijkstra	0.08	10854	5.8642	9
	A*	0.01	488	5.8642	9
3	Dijkstra	0.07	10499	6.0648	10
	A*	0.02	783	6.0648	10
4	Dijkstra	0.09	11352	7.5337	11
	A*	0.01	603	7.5337	11

Tabla 5.4: Comparativa entre Dijkstra y A* sobre los cuatro pares seleccionados.

Tiempo de cálculo. En el primer caso ambos algoritmos son bastante rápidos y la diferencia de tiempo es despreciable. Sin embargo, en los casos 2, 3 y 4, A* es hasta 9 veces más rápido que Dijkstra. Esta diferencia se debe a la reducción en el número de vértices procesados por A*, como se detalla a continuación.

Vértices explorados. La diferencia más llamativa entre ambos algoritmos se encuentra en esta métrica. En los casos presentados, A* explora desde 13 hasta 22 veces menos vértices que Dijkstra. Aunque la mayor diferencia en proporción se da en el caso 2, el primer caso es muy destacable: A* explora tan solo 14 vértices frente a los 296 explorados por Dijkstra. Las canciones de origen y destino se encuentran muy cercanas en el espacio de características y el camino óptimo no atraviesa aristas penalizadas, de modo que la heurística estima con precisión dónde está el destino y guía la búsqueda casi sin desviarse. En los casos más complicados, la heurística sigue siendo informativa, aunque menos precisa que en el primer caso, lo que hace que A* explore más vértices que en el caso 1, pero siempre muchos menos que Dijkstra.

Coste de la ruta y saltos. En los cuatro casos, se tiene que el coste de la ruta y el número de saltos son idénticos en ambos algoritmos. Estos resultados no son casualidad: si la heurística no fuese admisible, A* podría devolver un camino distinto (y, en general, peor) al de Dijkstra. Estos resultados validan de forma experimental la corrección de A*, es decir, que si se ejecuta A* con una heurística admisible, el algoritmo encuentra el camino óptimo.

En conclusión, Dijkstra es correcto por construcción y, gracias a la admisibilidad de la heurística, A* hereda esta optimalidad. Además, esta heurística permite mejorar la eficiencia, tanto en número de vértices explorados como en tiempo de cálculo. Cabe destacar que la base de datos utilizada en este trabajo consta de unas 26000 canciones, pero si se utilizara este modelo con una base de datos de gran tamaño (como el catálogo completo de

Spotify) la diferencia sería abismal. En este caso, Dijkstra resultaría excesivamente lento, demostrando la necesidad de la heurística implementada en A*. Por tanto, en el contexto de este trabajo, A* es el algoritmo más adecuado para construir playlists con transiciones suaves a partir del grafo de canciones.

5.6– Comparativa global

Una vez analizados los algoritmos por separado, conviene resumir los resultados para obtener una visión global que permita comparar el comportamiento de BFS, Dijkstra y A* sobre el grafo de canciones. De esta forma, se podrá decidir qué algoritmo utilizar según cada métrica. La siguiente tabla recoge los datos obtenidos a lo largo del capítulo.

nº	Algoritmo	Tiempo (s)	Vértices explorados	Coste de la ruta	Salto (Canciones)
1	BFS	0.00	236	4.8810	4
	Dijkstra	0.01	296	3.4649	4
	A*	0.01	14	3.4649	4
2	BFS	0.01	11633	7.4189	8
	Dijkstra	0.08	10854	5.8642	9
	A*	0.01	488	5.8642	9
3	BFS	0.01	10282	10.6151	8
	Dijkstra	0.07	10499	6.0648	10
	A*	0.02	783	6.0648	10
4	BFS	0.01	7417	8.2295	8
	Dijkstra	0.09	11352	7.5337	11
	A*	0.01	603	7.5337	11

Tabla 5.5: Comparativa entre BFS, Dijkstra y A* sobre los cuatro pares seleccionados.

A partir de los resultados que se muestran en la tabla 5.5, se observa que cada algoritmo destaca en una métrica diferente. BFS minimiza el número de saltos entre canciones, mientras que Dijkstra y A* minimizan el coste del camino encontrado. En cuanto a eficiencia, A* explora menos vértices que el resto de algoritmos en todos los casos, y su tiempo de cálculo es similar al de BFS y mucho menor que el de Dijkstra. En la siguiente tabla, se resume qué algoritmo es más adecuado según el criterio elegido.

Criterio	Algoritmo sugerido
Mínimo tiempo de cálculo	BFS o A*
Mínimo número de vértices explorados	A*
Mínimo coste de la ruta	Dijkstra o A*
Mínimo número de saltos	BFS

Tabla 5.6: Algoritmo más adecuado según el criterio elegido

Además del estudio del rendimiento de cada algoritmo, a partir de los resultados estudiados se pueden obtener conclusiones más generales.

Por una parte, la comparación entre los algoritmos que minimizan el coste (Dijkstra y

A*) y el que minimiza el número de saltos (BFS) refleja de forma cuantitativa el efecto del modelo de penalizaciones diseñado en el Capítulo 3. Los primeros, al utilizar los pesos del grafo, encuentran rutas musicalmente más coherentes, aunque más largas. Sin embargo, el segundo, al ignorar los pesos, puede producir transiciones más bruscas con tal de minimizar el número de saltos. Esto confirma que las restricciones de género impuestas guían la búsqueda hacia transiciones más naturales.

Por otra parte, la comparación entre Dijkstra y A* muestra la importancia de usar una heurística admisible, ya que de esta forma A* hereda la optimalidad de Dijkstra y la combina con la rapidez de BFS. Por tanto, A* permite encontrar el camino óptimo con gran eficiencia.

En el contexto de este trabajo (construcción de playlists que muestran la transición entre dos canciones), A* es el algoritmo más adecuado: encuentra el camino óptimo según el modelo descrito en el Capítulo 3 y lo hace en un tiempo equivalente al de BFS, un algoritmo de búsqueda no informada. BFS resulta útil únicamente si el objetivo es minimizar el número de saltos entre canciones, aunque esto pueda suponer que las transiciones sean bruscas. Sin embargo, Dijkstra solo destacaría si no se tuviera una heurística admisible, ya que, con la heurística seleccionada (o cualquier otra admisible), A* supera siempre a Dijkstra en eficiencia.

5.7— Aplicación interactiva con Streamlit

A lo largo de este capítulo se han estudiado los diferentes algoritmos sobre cuatro pares de canciones concretos. Para poder extender este análisis a cualquier par de canciones del grafo, se ha creado una aplicación interactiva con la librería **Streamlit**. Para implementar dicha aplicación, se han utilizado dos archivos: `algoritmos.py` y `aplicacion_interactiva.py`. Por una parte, el documento `algoritmos.py` recoge las versiones finales de los tres algoritmos (BFS, Dijkstra y A*). Por otra parte, el documento `aplicacion_interactiva.py` carga el grafo y los vectores de características, importa y ejecuta los algoritmos y genera la interfaz. Esta separación se ha hecho por claridad y para poder modificar únicamente los algoritmos si se quisiera hacer alguna mejora.

Al ejecutar el comando (`streamlit run aplicacion_interactiva.py`), la aplicación abre una pestaña en el navegador con la interfaz mostrada en la imagen 5.1.

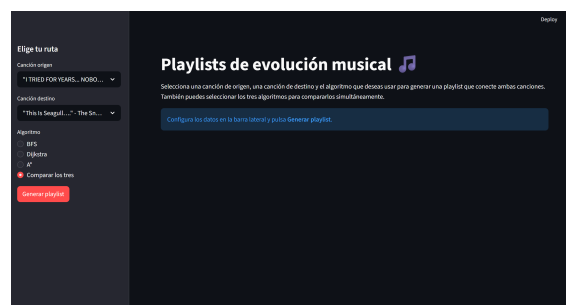


Figura 5.1: Interfaz inicial

Se observa que, a la izquierda, el usuario dispone de los siguientes controles:

- Dos desplegables para elegir la canción de origen y la canción de destino entre todas las canciones del grafo.
- Un selector para elegir el algoritmo que se desea ejecutar: BFS, Dijkstra, A* o los tres algoritmos a la vez.
- Un botón “Generar playlist” para comenzar la búsqueda.

Una vez pulsado el botón, pueden ocurrir diferentes situaciones. Si se ha seleccionado la misma canción como origen y como destino, se mostrará el mensaje “La canción de origen y destino son la misma”. Si se ha seleccionado una canción que no aparece en el grafo, se mostrará el mensaje “Alguna de las canciones no se encuentra en el grafo”. En caso de que se hayan elegido de forma adecuada las canciones de origen y destino, se mostrarán las cuatro métricas del algoritmo (tiempo, vértices explorados, coste de la ruta y saltos) y debajo la playlist que conecta la canción origen con la canción destino, donde cada canción aparece con su género en un color distintivo.

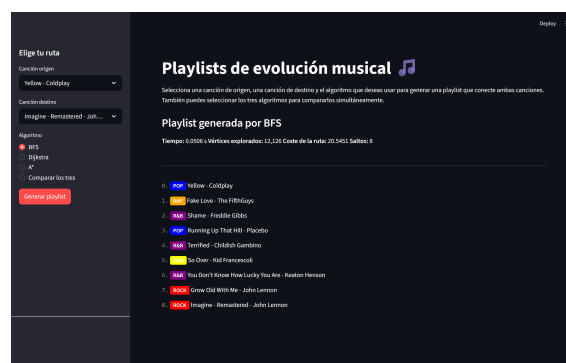


Figura 5.2: Playlist de BFS

Cuando el usuario selecciona “Comparar los tres”, la aplicación ejecuta los tres algoritmos y muestra los resultados en tres columnas paralelas, como se muestra en la imagen 5.3.

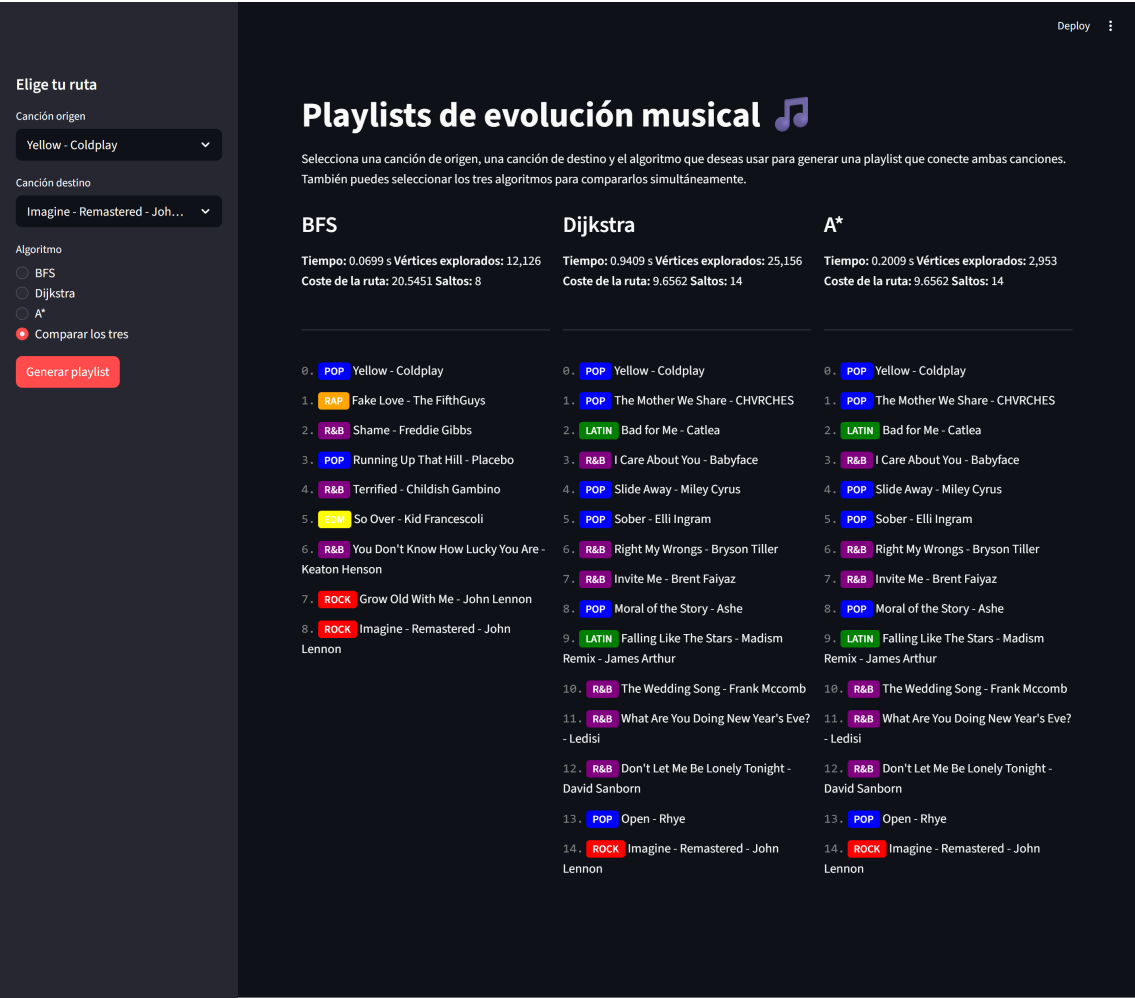


Figura 5.3: Playlists de los tres algoritmos

Esta aplicación complementa el análisis cuantitativo del capítulo, permitiendo al usuario comparar el comportamiento de BFS, Dijkstra y A* para cualquier par de canciones del grafo en las cuatro métricas estudiadas y observar las diferencias entre las playlists que genera cada algoritmo.

Conclusiones

En este trabajo se ha estudiado una aplicación de la teoría de grafos y los algoritmos de búsqueda más allá del ámbito tradicional de las redes de transporte: la generación de playlists musicalmente coherentes entre dos canciones. Para conseguirlo, se ha definido una base teórica sólida (espacios métricos, complejidad, pruebas de corrección de los algoritmos), para después poder utilizar técnicas aplicadas de la informática (procesamiento de datos, librerías de Python).

La construcción del grafo desde una base de datos de más de 30000 canciones ha exigido tomar decisiones de modelado y justificarlas debidamente (selección de los atributos, elección de la métrica, diseño de la matriz de compatibilidad de géneros y elección del parámetro k en el algoritmo k -NN). El resultado de estas decisiones ha sido un grafo conexo de 26229 vértices y grado medio de 13.78. Sobre este grafo se han implementado algoritmos de búsqueda que producen transiciones musicalmente coherentes.

De las comparativas que se han realizado se pueden extraer varias conclusiones sobre los algoritmos:

- **BFS** minimiza el número de saltos y es uno de los más rápidos en tiempo de ejecución, pero ignora los pesos de las aristas. En este contexto, esto se traduce en transiciones cortas pero bruscas.
- **Dijkstra** tiene en cuenta los pesos y devuelve siempre la playlist de menor coste, lo que se traduce en transiciones más suaves entre canciones musicalmente próximas. Sin embargo, tiene un coste computacional mayor, ya que explora todos los vértices cuyo coste acumulado no supera al del destino.
- **A*** hereda la optimalidad de Dijkstra gracias a la admisibilidad de la heurística y reduce sustancialmente el número de vértices explorados y el tiempo de cálculo. Todo esto lo convierte en el algoritmo más adecuado para la aplicación de este trabajo.

Además de las comparaciones entre algoritmos, este trabajo también valida el sistema de penalizaciones de género. Las penalizaciones de género han resultado efectivas a la hora de crear transiciones más naturales, mostrando que, aunque el grafo ha sido construido sobre características numéricas, es capaz de capturar parcialmente la percepción musical del oyente.

Por otra parte, el trabajo presenta varias líneas de mejora interesantes:

- Enriquecer el vector de características de cada canción incluyendo los atributos key y mode que en este trabajo se han descartado. El atributo key indica la tonalidad general estimada de la canción, mientras que el atributo mode indica su modalidad (mayor o menor). Ambos están estrechamente ligados a la percepción armónica de la música, por lo que añadirlos permitiría calcular conexiones más precisas entre canciones y, por tanto, transiciones más suaves.
- Refinar la matriz de compatibilidad de géneros añadiendo los subgéneros de las canciones.
- Ampliar este modelo a una base de datos más amplia para hacer más notorias las diferencias de eficiencia entre algoritmos.
- Validar la calidad de las playlists generadas mediante un estudio con usuarios reales, complementando las métricas cuantitativas con evaluaciones subjetivas.

En conclusión, el trabajo muestra cómo conceptos matemáticos pueden aplicarse a problemas reales distintos a los habituales, además de dejar abierta la posibilidad de futuras mejoras y extensiones. Se ha conseguido el objetivo planteado: al reformular la generación de transiciones musicales como un problema de camino más corto en un grafo, es posible resolverlo de forma eficiente y obtener playlists coherentes.

Bibliografía

- [1] AITUDE. Basics of k-nearest neighbor algorithm. <https://www.aitude.com/basics-of-k-nearest-neighbor-algorithm/>, 2023. Accedido: 25 de marzo de 2026.
- [2] CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., AND STEIN, C. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [3] ERICKSON, J. *Algorithms*. Independent, 2019.
- [4] HART, P. E., NILSSON, N. J., AND RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4, 2 (1968), 100–107.
- [5] JAMES, G., WITTEN, D., HASTIE, T., TIBSHIRANI, R., AND TAYLOR, J. *An Introduction to Statistical Learning with Applications in Python*. Springer Texts in Statistics. Springer Nature, 2023.
- [6] JOE BEACH. 30000 spotify songs. <https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs>, 2023. Accedido: 13 de abril de 2026.
- [7] PÉREZ AGUILA, R. *Una introducción a las matemáticas discretas y teoría de grafos*. Lulu Press, 2013.
- [8] RUSSELL, S., AND NORVIG, P. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [9] SCIKIT-LEARN DEVELOPERS. Preprocessing data: Standardization or mean removal and variance scaling. <https://scikit-learn.org/stable/modules/preprocessing.html#standardization-or-mean-removal-and-variance-scaling>, 2026. Accedido: 14 de abril de 2026.
- [10] TORO BONILLA, M. *Análisis y diseño de algoritmos y tipos de datos*. Colección Manuales de Informática del Instituto de Ingeniería Informática, n.º 3. Editorial Universidad de Sevilla, Sevilla, 2023.
- [11] VRAJITORU, D., AND KNIGHT, W. *Practical Analysis of Algorithms*. Undergraduate Topics in Computer Science. Springer, New York, 2014.